img/logo_unibo.png

Department of Physics and Astronomy "A. Righi"

Theoretical Physics

Complex Networks

Lectures held by
Prof. Daniel Remondini

Notes by
Gioele Mancino

Academic Year 2025/2026

Contents

Introduction

1 | Definitions

In this chapter, we will cover the basic definitions and concepts related to complex networks. We will start with fundamental graph concepts, then explore various network representations, discuss graph isomorphism and permutation, and finally delve into connectedness, distances, subgraphs, planarity, homomorphism, network reduction, and spectral analysis of networks.

1.1 | Basic Graph Concepts

We are going to start with the most basic definition of a graph, which serves as the foundation for all subsequent concepts in network theory. To introduce the concept of a graph, we will use the following definition:

Definition 1.1 (Graph)**.** A graph $G(V, L)$ is a finite nonempty set V of p points (vertexes or nodes) and a set of k couples of distinct points L (links, lines, or edges).

From this definition, the concept of a graph is quite general and can be applied to a wide range of systems. The nodes can represent various entities such as people, computers, or proteins, while the links can represent relationships, interactions, or connections between these entities. The flexibility of this definition allows us to model complex systems in various fields, including social networks, communication networks, biological networks, and many others. There are some general rules that we typically follow when working with graphs, which are important to keep in mind as we explore more complex network structures.

Remark (General Rules).

- *In general, networks have no loops (self-connections).*
- *There are no multigraphs, meaning no multiple links between the same nodes.*
- *We can, however, consider multiple links as weights or as multiple layers (multiplex).*

1.1.1 | Directed and Undirected Graphs

Directed graphs (DiGraphs) are graphs with directed edges, known as arcs, indicating a "**from-to**" **relationship**. They are used to describe hierarchical relationships, fluxes, or the passing of messages and goods. Undirected graphs contain only symmetric links and are used to describe symmetrical relations such as friendship, chemical bonds, or statistical correlation. Note that directed graphs can be symmetrized; while this loses some information, it allows the use of simpler properties and algorithms.

1.1.2 | Walk, Trail, Path, and Cycle

Let us consider a graph with nodes i and j. We can define different types of connections between these nodes:

- **Walk**: A node and edge sequence between two nodes.
- **Trail**: A walk with all distinct links.
- **Path**: A walk with all distinct nodes and links.
- **Cycle**: A closed path with $N_c \geq 3$ nodes.

This classification is important for understanding the structure of networks and for defining concepts such as connectedness and distances. Different algorithms are used to find walks, trails, paths, and cycles in a graph, which can be crucial for applications like network traversal, routing, and community detection.

1.2 | Network Representations

It is important to note that a graph can be represented in various ways, each with its own advantages and disadvantages. The choice of representation often depends on the specific application and the properties of the graph being studied.

1.2.1 | Adjacency Matrix

The most common representation of a graph is the adjacency matrix, which is a square matrix that encodes the presence or absence of links between nodes. For unweighted graphs, the entries are binary (0 or 1), while for weighted graphs, the entries can take on a range of values corresponding to the weights of the links.

Definition 1.2 (Adjacency Matrix)**.** The adjacency matrix A is an $N \times N$ matrix representing N nodes. For unweighted networks, $A_{ij} = 1$ indicates a topological link from node i to node j, with $A_{ii} = 0$ to indicate no loops. For weighted networks, $A_{ij} = w_{ij}$ where w_{ij} is the weight of the link.

Directed Graph
Consider the following directed graph with 6 nodes, we can represent it with the following adjacency matrix:

img/generic_adjacency.png

Figure 1.1: A directed graph and its corresponding non-symmetric adjacency matrix.

Undirected Graph
Consider the following undirected graph with 6 nodes, we can represent it with the following adjacency matrix:

img/symmetric_adjacency.png

Figure 1.2: An undirected graph and its corresponding symmetric adjacency matrix.

Remark (Logical Operators on Adjacency Matrices). *We can apply element-wise logical operators on adjacency matrices:*

- *Network symmetrization uses a logical OR:* $A' = \max(A, A^T) = A \cup A^T$.
- *Identifying reciprocal links uses a logical AND:* $A'' = \min(A, A^T) = A \cap A^T$.
- *Identifying strictly directed links uses logical XOR:* $XOR(A, A^T)$.

1.2.2 | Weighted Network

In a weighted network, the adjacency matrix contains values that represent the strength of the connections between nodes:

$$A_{ij} = w_{ij} \text{ link from node } i \text{ to node } j \text{ with weight } w_{ij}.$$

This can be interpreted in various ways depending on the context, such as the number of interactions, the intensity of relationships, or the capacity of links.

It can be seen as a representation of a **multigraph** with N multi-edges: $w_{ij} = N$. If all weights are positive, we have useful theorems for matrices with positive values (Perron-Frobenius theorem) and we can connect to "physical" situations (transition probabilities of diffusion processes) Negative weights can be meaningful too: in **signed networks** the sign of the weight can represent different types of relationships or interactions, such as:

- **Friend-enemy**: in social networks, positive weights can represent friendships while negative weights can represent enmity.
- **Attraction-repulsion**: in physical systems, positive weights can represent attractive forces while negative weights can represent repulsive forces.
- **Ferromagnetic-antiferromagnetic**: in spin systems, positive weights can represent ferromagnetic interactions while negative weights can represent antiferromagnetic interactions.

One can also operate a weighted matrix symmetrization:

- Average symmetrization: $A' = \frac{1}{2}(A + A^T)$.
- Random matrix symmetrization: $A' = \frac{1}{\sqrt{2}}(A + A^T)$.

1.2.3 | Incidence Matrix

We can also represent a graph using an incidence matrix, which is an alternative to the adjacency matrix. The incidence matrix is particularly useful for certain types of analyses, such as flow problems and network dynamics.

Definition 1.3 (Incidence Matrix)**.** The incidence matrix I of a graph $G(V, L)$ is an $L \times V$ matrix (number of links by number of vertexes). It contains -1 for outgoing connections and $+1$ for ingoing connections:

$$\text{if } A_{ij} \neq 0, \quad I_{Ni} = \mp 1, \quad I_{Nj} = \pm 1.$$

It can be interpreted as a "generalized derivative" on the graph, giving the oriented difference between values on adjacent nodes. If there is no preferred direction, we can use an unsigned incidence matrix with $I_{Ni} = 1$ for all links. It has to be read as if each link is a vector pointing from the source node to the target node, and the incidence matrix encodes the direction and magnitude of these vectors.

Thus the incidence matrix can be used to analyze the flow of information or resources through the network, and it is particularly useful for studying directed graphs and their properties. The incidence matrix can also be used to derive the adjacency matrix through the relation $A = I^T I - D$, where D is a diagonal matrix that accounts for the degree of each node.

Generalized Derivative

The incidence matrix can be interpreted as a generalized derivative on the graph, giving the oriented difference between values on adjacent nodes. The incidence matrix allows us to compute the net flow into or out of each node, which is essential for understanding the dynamics of processes.

Given a vector of values $\mathbf{f}_i$ for each node, the incidence matrix I applied on the vector gives:

$$\sum_j I_{ij} \cdot \mathbf{f}_j = \Delta_i,$$
$$\Delta_i = \mathbf{f}_i - \mathbf{f}_j \forall \text{adjacent nodes},$$

where Δ_i represents the net flow into or out of node i based on the values of its neighbors. This interpretation is crucial for analyzing diffusion processes, random walks, and other dynamic phenomena on networks.

Geometrically, a network acts as a spatial structure: while a regular lattice discretizes Euclidean space, a complex graph represents a more intricate manifold. However, one must be careful, as the properties of this discrete manifold do not necessarily hold in the continuous limit.

1.2.4 | Sparse Representations

For sparse graphs where the number of links is much smaller than V^2, other representations are preferred:

- **Edge Matrix**: An $L \times 2$ matrix that stores single links instead of the full matrix. It is more efficient for sparse graphs, but it does not allow for fast access to neighbors. It is way less memory-consuming for sparse graphs:

  $$L = o(V^2) \quad \Longrightarrow \quad 2 \times L \ll V^2.$$

  It is useful for algorithms that need to iterate over links, such as those used in community detection or clustering.

- **Adjacency List**: Each line describes a node and contains the indices of its arrival nodes. This is particularly useful for diffusion processes, such as simulating random walks on a graph, but it has a higher memory cost than the edge matrix, since it requires storing the neighbors of each node and the same link is stored twice (once for each node it connects).

Example. Sparse Graph Representations Let us consider a sparse graph with 6 nodes and 4 links. The adjacency matrix representation would require a 6×6 matrix, which contains many zeros, while the edge matrix would only require a 4×2 matrix to store the links. The adjacency list would also be more efficient in this case, as it would only need to store the neighbors of each node, rather than the full matrix. Explicitly:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}, \quad I = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad E = \begin{pmatrix} 1 & 2 \\ 2 & 3 \\ 4 & 5 \\ 5 & 6 \end{pmatrix}, \quad L: \begin{matrix} 1 \to \{2\} \\ 2 \to \{1,3\} \\ 3 \to \{2\} \\ 4 \to \{5\} \\ 5 \to \{4,6\} \\ 6 \to \{5\} \end{matrix}$$

where A is the adjacency matrix, I is the incidence matrix, E is the edge matrix, and L is the adjacency list. As we can see, the adjacency matrix contains many zeros, while the edge matrix and adjacency list are more compact representations of the same graph.

Random Walks and Diffusion Processes

The choice of representation can significantly impact the efficiency of algorithms for simulating random walks and diffusion processes on networks. For example, the adjacency list allows for efficient access to neighbors, which is crucial for simulating random walks, while the edge matrix can be more efficient for algorithms that need to iterate over links, such as those used in community detection or clustering.

To emulate a **random walk on a graph**, we can use the adjacency list to quickly access the neighbors of each node and determine the probabilities of transitioning from one node to another. This is particularly important for large, sparse graphs where the number of links is much smaller than V^2, as it allows for efficient memory usage and faster computations. The steps to follow for simulating a random walk on a graph using the adjacency list are as follows:

1. Start at a randomly selected node (or a specific node if desired).
2. At each step, use the adjacency list to find the neighbors of the current node.
3. Randomly select one of the neighbors to transition to, based on the weights of the links (for a pure random walk we have to consider a uniform distribution, otherwise we could use the link weights/biases etc.).
4. Repeat the process for a specified number of steps or until a certain condition is met (e.g., reaching a target node).

This method allows us to efficiently simulate random walks and diffusion processes on large, sparse networks, which is essential for understanding the dynamics of complex systems.

1.3 | Graph Isomorphism and Permutation

We are now going to discuss the concept of graph isomorphism, which is a fundamental notion in graph theory that describes when two graphs are essentially the same, even if they are represented differently.

Definition 1.4 (Graph Isomorphism)**.** An isomorphism is a 1-1 correspondence between vertexes that preserves the link structure. It corresponds to a node index permutation.

This means that two graphs are isomorphic if there exists a permutation of the node labels of one graph that results in the adjacency matrix of the other graph. In other words, the two graphs have the same structure, but their nodes may be labeled differently. We have then to digress on the concept of node permutation and permutation matrix, which is a fundamental operation in graph theory that allows us to rearrange the nodes of a graph without changing its structure.

A permutation of the nodes of a graph can be represented by a permutation matrix, which is a square binary matrix that has exactly one entry of 1 in each row and each column, and 0s elsewhere. The permutation matrix can be used to rearrange the rows and columns of the adjacency matrix of a graph, effectively permuting the nodes. There exist $N!$ possible permutations of the nodes, which means that there are $N!$ different permutation matrix realising the isomorphism: each $N \times N$ network can be represented by $N!$ different adjacency matrices, all isomorphic.

The properties of the permutation matrix are as follows:

$$\begin{aligned} P \cdot P^T = \mathbb{I} \implies P^T = P^{-1} \quad & \text{(orthogonality)} \\ P \cdot A \cdot P^T = A' \quad & \text{(adjacency matrix transformation)} \\ |P| = \pm 1 \quad & \text{(determinant)} \end{aligned}$$

Remark (Re-indexing). *Node permutation represents node reordering, which is a common operation in graph algorithms. It allows us to rearrange the nodes of a graph without changing its structure, which can be useful for various applications such as community detection, clustering, and visualization. However, it is important to note that while node permutation can change the appearance of the adjacency matrix, it does not change the underlying properties of the graph, such as its degree distribution, clustering coefficient, or path lengths. Thus node re-indexing has to be made after gaining insights on the graph structure, thus it can be based on:*

- ***A priori information****: we can have information directly about the network, due to ontological knowledge of the system (e.g. known community structure or node attributes) or intrinsic metric properties of the graph (e.g., spatial networks, node proximity).*
- ***Network-based information****: we can have information about the network structure, such as node degree, centrality measures, or community structure, which can be used to reorder the nodes in a way that reveals patterns or simplifies the analysis.*

These re-indexing strategies can help us to better understand the structure of the graph and to identify important features or patterns that may not be immediately apparent in the original representation. The **visualization** of a network can be significantly improved by reordering the nodes based on their properties or the structure of the graph, which can help to reveal hidden patterns and relationships between nodes (a chain network can be visualized as a sparse one if we make poor re-indexing choices).

Example (Protein and DNA Contact Maps). A network representation of peptide bonds serves as a discretization of a 3D distance matrix: a fully connected graph with weights given by the spatial distance between amino acids. The re-indexing of nodes can be based on the spatial proximity of amino acids, which can help to reveal patterns in the contact map and provide insights into the structure and function of the protein.

Spatial closeness acts as an “intrinsic” metric for the graph. One can use the re-indexing in order to exploit different properties of the graph, such as symmetry or centrality measures, study spatial closeness, revealing the backbone of the network, or identify communities.

DNA 3D structure can also be represented as a similar network, where nodes represent nucleotides and links represent spatial proximity or interactions between them. Re-indexing based on spatial proximity can help to reveal patterns in the contact map and provide insights into the structure and function of the DNA molecule (the hi-c technique is used to obtain such contact maps).

1.4 | Connectedness

Studying the connectedness of a graph is crucial for understanding its structure and the dynamics of processes that occur on it. Connectedness refers to the extent to which nodes in a graph are connected to each other, and it can be classified into different types based on the nature of the connections.

1.4.1 | Undirected Graphs

An undirected graph is **connected** if it is not divided into two or more non-communicating parts: from every node there is a path to every other node. If there are multiple disconnected components, the graph is called **disconnected**. Algebraically, this means there is no permutation that reduces the adjacency matrix into diagonal blocks, it is not isomorphic to a block diagonal matrix

$$PAP^{-1} = \begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix}.$$

The number of blocks corresponds to the number of disconnected components. Any matrix which can be transformed into a block form (diagonal or upper triangular) is called **reducible**, while a matrix that cannot be transformed into such a form is called **irreducible**.

It is important to notice that from the same dataset more then one graph can be constructed, depending on the definition of the links; even from the same adjacency matrix, different re-indexing can lead to different visualizations of the same graph, which can reveal different patterns and structures. But before any analysis, it is crucial to check the connectedness of the graph, as many properties and algorithms assume that the graph is connected. Some network properties rely on the assumption of connectedness, other than the components number and size, so it is a crucial information to have before starting any analysis.

1.4.2 | Directed Graphs

In a directed graph, not every node is necessarily reachable from any other. After a **block matrix reduction**, we are often left with a block upper triangular matrix, where the blocks on the diagonal represent strongly connected components

$$PAP^{-1} = \begin{pmatrix} A_1 & B \\ 0 & A_2 \end{pmatrix}, \quad A_1 \xrightarrow{B} A_2, \quad A_1 \not\leftarrow A_2.$$

In this situation, we can say that block A_1 is **upstream** of block A_2, while block A_2 is **downstream** of block A_1. The presence of the block B indicates that there are connections from nodes in block A_1 to nodes in block A_2, but not vice versa. This structure can have important implications for the dynamics of processes on the graph, such as diffusion or information flow.

In a directed graph, connectedness does not guarantee that every node is reachable from any other. It is classified into three types:

- **Strongly connected**: Every node is connected to another by a path, implying the existence of cycles.
- **Unilaterally connected**: For every two nodes, there is at least one path (i.e. in one direction, as for the absorbing states seen before).

- **Weakly connected**: Every node is connected to another by at least a semipath.

Semi-paths, semi-walks etc. are defined as paths, walks etc. where it is necessary to traverse edges in an unspecified direction (we may need to reverse the direction of one or more links to complete the path). The presence of cycles in a directed graph is a key factor in determining whether the graph is strongly connected, as cycles allow for bidirectional connectivity between nodes.

We can also write three theorems about the connectedness of directed graphs:

- Strongly connected implies that exists one spanning closed walk.
- Unilaterally connected implies that exists one spanning walk.
- Weakly connected implies that exists one spanning semiwalk.

All these theorems can be proved by contradiction, by assuming that there is no spanning walk and showing that this leads to a contradiction with the definition of connectedness.

`img/connectedness_examples.png`

Figure 1.3: Examples of different types of connectedness in directed graphs. The left graph is weakly connected (since there is a semipath between every pair of nodes), the middle graph is unilaterally connected (since there is at least one path between every pair of nodes), and the right graph is strongly connected (since every node is reachable from every other node with a path).

1.5 | Distances and Shortest Paths

We are now going to discuss the concept of distance in graphs, which is a fundamental notion that allows us to quantify how far apart nodes are from each other. The distance between two nodes can be defined in various ways, depending on the context and the type of graph being studied.

1.5.1 | Node Distance

The distance between two nodes is defined as the shortest path between them. There are different ways to define distance, depending on the context and the type of graph:

- **Unweighted distance**: The distance is the number of links in the shortest path between two nodes.
- **Weighted distance**: The distance is the sum of the weights of the links in the shortest path between two nodes.

Let us study more in detail each case.

Unweighted Distance

In unweighted networks, the distance between two nodes is simply the number of links in the shortest path connecting them. This is often referred to as the **geodesic distance**. The geodesic distance can be defined as

- Isolated nodes have infinite distance from any other node: $D(u, v) = \infty$ if there is no path between nodes u and v.
- Adjacent nodes have a distance of 1: $D(u, v) = 1$ if there is a link between nodes u and v.
- A node is at distance 0 from itself: $D(u, v) = 0$ if $u = v$.
- The distance between two nodes is the minimum number of links in any path connecting them: $D(u, v) = \min\{k : \text{there exists a path of length } k \text{ from } u \text{ to } v\}$.

Note that topologically, if we are defining a distance on a symmetric graph, we are defining a metric space, since the distance satisfies the properties of a metric:

1. **Positive semidefiniteness**: $D(u, v) \geq 0$ for all nodes u and v, and $D(u, v) = 0$ if and only if $u = v$.
2. **Symmetry**: $D(u, v) = D(v, u)$ for all nodes u and v.
3. **Triangle inequality**: $D(u, w) \leq D(u, v) + D(v, w)$ for all nodes u, v, and w.

One last remark is that one can define different types of distances, such as the distance between node embeddings, one with the overlap of n neighbors, or the distance between communities, which can be defined as the distance between their centroids in a suitable embedding space.

Weighted Distance

In weighted networks, nodes that are "closer" have a greater weight, and distance can be calculated using **harmonic composition** (similar to capacitors in series): if we have more path between two nodes, the shortest path is the one with the greatest weight. We can define the weighted distance as

$$d_{ij} = \sum_{i \to j} \frac{1}{w_{kl}},$$

where the sum is taken over all paths from node i to node j, and w_{kl} is the weight of the link between nodes k and l along the path. This definition allows us to capture the idea that stronger connections (higher weights) correspond to shorter distances, while weaker connections (lower weights) correspond to longer distances. If there is a zero-weight link, it can be interpreted as an infinite weight (i.e., no connection), which means that the distance between the nodes is infinite:

$$w = 0 \implies d \to \infty.$$

Example. The matrix of **Euclidean distances** d_{ij} between elements of a defined N-dim space, can be seen as a fully connected symmetric weighted network: every element is connected to every other element, and the weight of the link between two elements is given by the Euclidean distance between them. This representation can be useful for analyzing spatial relationships and clustering in data, as well as for visualizing high-dimensional data in a lower-dimensional space. For example, in a protein structure, we can represent the distances between amino acids as a weighted network, where nodes represent amino acids and links represent the spatial proximity between them. This can help us to understand the structure and function of the protein, as well as to identify important interactions between amino acids.

Depending on the context, we can also define the distance as a function of the weights $d_{ij} = f(w_{ij})$, where f is a suitable function that captures the relationship between weights and distances. For example, we could use an exponential decay function, or a logarithmic function, one based on gaussian kernels, etc. The choice of the function f would depend on the specific application and the properties of the data being analyzed.

1.5.2 | Shortest Path and Algorithms

The distance between two nodes equals the minimum number of links (geodesic) between them. It is useful to define some auxiliary concepts:

- **Eccentricity**: The maximum distance from a node to any other node in the graph.
- **Radius**: The minimum eccentricity among all nodes in the graph.
- **Diameter**: The maximum eccentricity among all nodes in the graph.
- **Average path length**: The average distance between all pairs of nodes in the graph.

The shortest path can be found using various algorithms, some of the most common ones include:

- **Dijkstra's algorithm**: This algorithm is used for *finding the shortest paths between nodes in a graph*, which may represent, for example, road networks. It works by iteratively selecting the node with the smallest tentative distance and updating the distances to its neighbors until the shortest path to the target node is found (one-to-all mechanism).

- **Floyd-Warshall algorithm**: This algorithm is used for *finding the shortest paths between all pairs of nodes in a graph*. It works by iteratively updating a distance matrix based on the shortest paths found so far, until all pairs of nodes have been considered (all-to-all mechanism).

Both of these algorithms are implemented efficiently in every SW language (such as Python, C, R, Matlab etc.) and can be used to compute shortest paths in large graphs. The choice of algorithm depends on the specific requirements of the problem, such as whether we need to find the shortest path between a single pair of nodes or between all pairs of nodes.

1.6 | Subgraphs and Special Network Topologies

We are going to discuss some special types of subgraphs and network topologies that are commonly studied in graph theory and network science. These structures can provide insights into the properties and dynamics of complex networks.

Definition 1.5 (Subgraph, Spanning Subgraph, Induced Subgraph)**.**

- **Subgraph**: A subset of a graph's nodes and links.
- **Spanning Subgraph**: A subgraph that contains all the nodes of the original graph (but not necessarily all the links).
- **Induced Subgraph**: A subset of the nodes of G with ALL the links of G that exist between those specific nodes.

`img/subgraphs_examples.png`

Figure 1.4: Examples of different types of subgraphs. G is the original graph, G_1 is an induced subgraph while G_2 is a spanning subgraph.

To extract the adjacency matrix of an induced subgraph S, you simply need to find the intersection of the rows and columns of the original matrix A that correspond exactly to the nodes in S. Practically, removing elements from the network translates to very basic matrix operations:

- **Node removal:** To remove a specific node i from the network, you just delete the entire i-th row and the i-th column from the adjacency matrix A.
- **Link removal:** To remove the connection between node i and node j, you simply set the matrix element a_{ij} to 0. If the graph is undirected (symmetric), you must also remove the reciprocal link by setting a_{ji} to 0.

1.6.1 | Complete Graphs, Complement, K-Cliques and Bipartite Graphs

Let us now discuss some special types of graphs that are commonly studied in graph theory and network science, including complete graphs, graph complements, k-cliques and bipartite graphs.

Definition 1.6 (Complete Graph)**.** A completely connected graph K_N with N nodes with all possible links is called a **complete graph**, meaning every node has a degree of $K = N - 1$:

$$\#l = \binom{N}{2} = \frac{N!}{2!(N-2)!} = \frac{N(N-1)}{2}.$$

Definition 1.7 (Graph Complement and Self-Complementary)**.** The **graph complement** is a network consisting of the exact same set of nodes, but with "opposite" connections. Specifically, two nodes are linked in the complement graph if and only if they were not linked in the original graph.

Algebraically, if A is the adjacency matrix of the original graph, the matrix of its complement can be calculated by applying $(A+1) \pmod 2$ to the elements (keeping the diagonal as zero).

Furthermore, a graph is defined as **self-complementary** if it is isomorphic to its own complement. This means that, despite the links being inverted, the new network shares the exact same structural topology as the original one.

Definition 1.8 (K-Clique)**.** A **K-clique** is a fully connected subgraph of G with degree K. Finding a clique of a certain size in a graph, known as the **clique problem**, is NP-complete. This means it is computationally infeasible, as the complexity grows more than a power of the number of nodes.

A strongly connected subgraph, such as a clique, can be seen as a **community** within the network, meaning its nodes are likely to share similar properties. While a single link in an adjacency matrix describes a simple 1-to-1 relationship, a clique can effectively represent an N-body relationship. Practical examples of such structures include protein complexes, chemical reactions, and other multi-body interactions.

Definition 1.9 (Bipartite Graph)**.** A graph that connects two different types of nodes between each other, but NOT between themselves. It is represented by an $M \times N$ rectangular adjacency matrix. In terms of matrix algebra, a bipartite graph is an imprimitive matrix of order 2.

Thus in a bipartite graph, we have two distinct sets of nodes, and links only exist between nodes from different sets. This structure is common in many real-world networks, such as social networks (e.g., people and their affiliations), biological networks (e.g., genes and diseases), and information networks (e.g., users and items in a recommendation system).

In a complete bigraph every node of one type is connected to every node of the other type. The number of links in a biclique is given by $L = M \times N$, where M and N are the number of nodes in the two sets.

The **adjacency matrix** of a bipartite graph can be represented in block form as follows:

$$A = \begin{pmatrix} 0 & B \\ B^T & 0 \end{pmatrix},$$

where B is an $M \times N$ matrix that represents the connections between the two sets of nodes. The upper left and lower right blocks are zero matrices, indicating that there are no connections between nodes of the same type.

`img/bipartite_graph_example.png`

Figure 1.5: An example of a bipartite graph and its corresponding block adjacency matrix; it is structured in a way that reflects the bipartite nature of the graph, with the off-diagonal blocks representing the connections between the two sets of nodes.

Primitivity and Imprimitivity

When representing a bipartite graph with an $(M+N)\times(M+N)$ square adjacency matrix, we can always find a node permutation that neatly separates the two sets of nodes. This rearrangement leads to a block matrix structure featuring zero blocks on the main diagonal and two non-diagonal blocks representing the cross-connections: there exists a permutation matrix P such that

$$PAP^{-1} = \begin{pmatrix} 0 & B^1_{mn} \\ B^2_{nm} & 0 \end{pmatrix}.$$

In terms of matrix algebra, this specific block structure means the bipartite graph corresponds to an **imprimitive matrix of order 2**. Conceptually, traversing this network results in a continuous "flip" in node values as you alternate between the two distinct sets. A **primitive matrix** instead corresponds to a graph where there is a path of any length between any two nodes, without the need for such a flip, indicating a more interconnected structure.

This topological and algebraic concept can be generalized to higher dimensions: an N**-partite graph** corresponds exactly to an N**-imprimitive matrix**. By applying the appropriate node permutation, its adjacency matrix can be organized into a "cyclic" N-block array. In this configuration, all nodes within a given block are connected *only* to nodes belonging to another specific block, and there are no connections between nodes within the same block. This structure leads to a continuous "flip" in node values as you traverse the network, cycling through the different blocks in a specific order.

`img/n_partite_graph_example.png`

Figure 1.6: An example of a n-partite graph and its corresponding block adjacency matrix; the adjacency matrix is structured in a way that reflects the n-partite nature of the graph, with the off-diagonal blocks representing the connections between the n sets of nodes.

Determining whether a matrix is primitive or imprimitive is crucial in network analysis. **Primitivity** is a fundamental algebraic property required to apply several advanced mathematical theorems to networks, most notably the Perron-Frobenius theorem.

Tensor Contraction

Given the rectangular adjacency matrix B of dimensions $M \times N$, we can perform a tensor contraction to obtain two distinct square networks that connect nodes of the same type. This is achieved through simple matrix multiplication:

- An $M \times M$ matrix calculated as $B \cdot B^T$.
- An $N \times N$ matrix calculated as $B^T \cdot B$.

In both of these resulting matrices, two nodes of the same type are linked if there is at least one path between them through a node of the other type in the original bipartite graph. If we treat the new networks as weighted, the link weight corresponds exactly to the number of common nodes of the other type they share.

Common examples of this projection include co-authorship networks (authors connected by shared papers), actors in the same movie, genes related to the same diseases, or drugs acting on the same diseases.

Remark (Directed Graph Symmetrization). *In addition to the standard topological symmetrization obtained by simply adding the matrix and its transpose ($A + A^T$), a directed graph represented by an $N \times N$ adjacency matrix A can be projected into two distinct symmetric graphs, B and C, using matrix multiplication.*

This process reveals hidden similarities between nodes based on their connection patterns:

- *By computing $B = A^T \cdot A$, we connect two nodes if they both point to at least one common target node. This is widely known as "bibliographic coupling"; for instance, two distinct research papers are linked if they cite the exact same reference in their bibliography.*
- *Conversely, by computing $C = A \cdot A^T$, we connect two nodes if they receive incoming links from at least one common source node. This represents "co-citation coupling", meaning two papers are linked if they are both cited by the exact same third paper.*

Example (The Human Disease Network). An example of a bipartite graph is the "diseasome" (e.g., OMIM database), a gene-pathology network. By contracting the matrix, we can obtain a gene-gene network based on shared diseases, or a pathology-pathology network based on shared genes.

Analyzing this projected disease network reveals several key medical insights:

- **Communities and connectivity:** Similar diseases (such as various types of cancers) share many genes and are therefore highly connected. As a result, diseases naturally form clustered communities based on organ-related proximity (e.g., heart, brain, or blood diseases) or functional similarities.
- **Unexpected topological results:** The network structure shows that, counterintuitively, not all organ-related diseases are connected. Furthermore, discovering unexpected links between seemingly unrelated diseases opens the door to **drug repurposing**—suggesting that a drug effective for one disease might also be applied to a connected, yet distinct, pathology.
- **Further analysis:** To extract deeper biological meaning, the network can be enriched by attaching additional "labels" to the nodes, such as disease mortality rates or the specific functional role of a gene within the cell.

1.6.2 | Trees and Stars

Let us now discuss some special types of tree-like structures that are commonly studied in graph theory and network science, including star networks and tree networks.

Star Network

The star network is a simple network topology where one central node is connected to all other nodes, which are not connected to each other. This structure resembles a star, with the central node acting as the **hub** and the other nodes as the **spokes**. The star network is characterized by its simplicity and efficiency in terms of communication, as all nodes can communicate with each other through the central hub. However, it also has a significant drawback: if the central node fails, the entire network becomes disconnected, as there are no alternative paths for communication between the peripheral nodes. The star network is often used in scenarios where a central authority or coordinator is needed, such as in certain types of organizational structures or in some communication networks. However, it is not suitable for large-scale networks or those that require high levels of redundancy and fault tolerance, due to its vulnerability to the failure of the central node.

Tree Network

A tree network is a hierarchical structure that consists of nodes connected in a **parent-child relationship**. It is a type of **acyclic graph** where each node has one parent and can have multiple children. The topmost node is called the root, and the nodes at the bottom are called leaves. Tree networks are commonly used in organizational structures, file systems, and decision-making processes. They allow for efficient data organization and retrieval but can become inefficient if the tree becomes too deep or unbalanced.

img/tree_network_example.png

It is characterized by its branching structure, where each node can have multiple child nodes, but only one parent node. This structure allows for a single path between any two nodes, making it easy to navigate and manage, there is no redundancy in the connections, which can lead to efficient communication and data transfer. However, it can also lead to bottlenecks if the root node or any intermediate nodes fail, as it can disrupt the entire network.

Figure 1.7: On the left, an example of a tree network, where the central node is the root, and the most external nodes are the leaves. Each node has one parent and can have multiple children, forming a hierarchical structure.

With multiple tree networks, it is possible to create a more complex structure, such as a **forest**, which consists of multiple trees that are not connected to each other. This can provide redundancy and improve the overall resilience of the network.

Example (Fractals as regular trees.). A fractal can be mathematically modeled as a regular tree network. These structures are highly ordered and characterized by specific topological properties: they are self-similar, modular, and deeply hierarchical.

To understand their spatial properties, we can define two metrics for a tree graph:

- **Graph Volume (V):** The total number of nodes in the network.
- **Graph Surface (S):** The number of leaf nodes (nodes with a degree of 1).

In standard Euclidean geometry (or regular lattices), as an object grows, its surface-to-volume ratio tends to zero ($\lim_{N\to\infty} \frac{S}{V} = 0$). However, fractal networks behave very differently: the number of leaves grows proportionally to the total number of nodes, meaning the surface is directly comparable to the volume ($V \simeq S$). In practical terms, a huge fraction of a fractal tree's nodes are leaves.

1.6.3 | Directed Acyclic Graph (DAG)

A directed acyclic graph (DAG) is a type of graph that consists of nodes connected by directed edges, where there are no cycles. In a DAG, each edge has a direction, indicating the flow of information or dependencies between nodes. DAGs are commonly used in various applications, such as scheduling tasks, representing dependencies in software development, and modeling hierarchical relationships. It can be used to represent complex systems where there are multiple levels of dependencies, without the risk of creating cycles that can lead to infinite loops or deadlocks.

With DAGs we can represent more complex relationships than trees, as they allow for multiple paths between nodes, establishing an effective hierarchy. However, they can also be more difficult to manage and navigate, especially as the number of nodes and edges increases. DAGs are often used in scenarios where there are multiple dependencies or relationships between entities, such as in project management, data processing pipelines, and knowledge representation.

img/GOTM_analysis_DAG_example.png

Figure 1.8: An example of a directed acyclic graph (DAG) on the GOTM analysis (which demonstrated that more specific biological processes, such as immune response, regulation of hydrolase activity, peptide transport and JAK-STAT cascade, were significantly modulated by the microbiota; the number of observed genes in a particular biological process is indicated by "n"). The edges have a direction and there are no cycles. The DAG structure allows for multiple paths between nodes, establishing an effective hierarchy.

For networks with these properties, the adjacency matrix is **upper triangular**, which means that all the entries below the main diagonal are zero. This is because in a DAG, there are no cycles, and therefore, there can be no edges that point back to previous nodes. The upper triangular structure of the adjacency matrix reflects the hierarchical nature of the DAG, where nodes can only have edges pointing to nodes that come after them in the hierarchy. Through this matrix, we can represent the structure of the DAG and analyze its properties as a linear algebra problem, such as finding the longest path or determining the reachability of nodes.

1.7 | Planarity and Homomorphism

Planarity and homomorphism are important concepts in graph theory that help us understand the structure and properties of networks. They provide insights into how networks can be represented and analyzed.

1.7.1 | Planarity and Genus

A graph is defined as **planar** if it can be drawn on a two-dimensional plane without any of its links (edges) crossing each other.

In topological terms, a planar graph has a **genus** of 0. The genus of a graph represents the minimum number of "holes" (or handles on a mathematical manifold, like a torus or a donut) that a surface must have so that the graph can be drawn on it without any intersecting links.

Because links cannot cross, planar graphs face strict structural limitations and must be relatively sparse. Specifically, there is a rigid upper bound: a planar graph with N nodes can have at most $3N - 6$ links, meaning the number of links scales linearly, $O(N)$.

To determine if a graph is planar, we rely on **Kuratowski's Theorem**. This theorem states that a graph is planar if and only if it does not contain any subgraphs that are homomorphic (topologically equivalent) to K_5 (the complete graph on 5 nodes) or $K_{3,3}$ (the complete bipartite graph with two sets of 3 nodes).

1.7.2 | Graph Homomorphism

A **graph homomorphism** is a mathematical mapping f from a network G (with N nodes) to a smaller network G' (with $M < N$ nodes) that rigorously preserves node adjacency.

Algebraically, if two nodes u and v are connected in the original graph ($u \sim v$), their mapped counterparts must be connected in the new graph ($f(u) \sim f(v)$):

$$G' = f(G) \quad \text{with} \quad f : G \to G' \quad \Big| \quad u \sim v \implies f(u) \sim f(v).$$

Practically, a homomorphism acts as a **dimensionality reduction** for a graph. It simplifies the network by drastically reducing the number of nodes while retaining its essential connectivity patterns and macroscopic characteristics. A classic example is mapping a large network of individual nodes into a smaller homomorphic network of **communities**: in this new graph, each super-node represents an entire community, and the links show how different communities interact with one another.

1.8 | Network Reduction and Filtering

We are often interested in reducing the complexity of a network by filtering out less important links, which can help us to focus on the most relevant connections and to analyze the structure of the network more effectively. This process can be done using various methods, such as minimum spanning trees (MST) or link selection rules based on global or local criteria.

1.8.1 | Minimum Spanning Tree (MST)

Let us define a spanning tree as a subgraph of a connected graph that includes all the vertices of the original graph and is a tree (i.e., it is connected and has no cycles) with a special property.

Definition 1.10 (Min/Max Spanning Tree)**.** A minimum/maximum spanning tree (MST) is a subset of the edges of a connected, weighted graph that connects all the vertices together without any cycles and with the minimum/maximum possible total edge weight. It is a method for finding the most efficient way to connect all the nodes in a graph while minimizing/maximizing the total cost: it finds the backbone of the network.

The MST can be used in various applications, such as designing efficient communication networks, clustering data, and solving optimization problems. There are several algorithms for finding the MST, including Prim's algorithm and Kruskal's algorithm, which are based on greedy strategies to build the tree incrementally by selecting the edges with the smallest weights.

Kruskal's Algorithm Kruskal's algorithm is a popular method for finding the minimum spanning tree (MST) of a connected, weighted graph. The algorithm works by sorting all the edges of the graph in non-decreasing order of their weights and then adding edges to the MST one by one, as long as they do not form a cycle. The process continues until there are $V-1$ edges in the MST, where V is the number of vertices in the graph. To resume:

1. Start: single node and list of all links (edges);
2. Sort all edges in non-decreasing order of their weights;
3. Add edges to the MST one by one, ensuring no cycles are formed;
4. Continue until there are $V-1$ edges in the MST, where V is the number of vertices.

Kruskal's algorithm is efficient and can be implemented using a disjoint-set data structure to keep track of the connected components of the graph, ensuring that cycles are not formed when adding edges to the MST.

Prim's Algorithm [...]

Example (Pearson Correlation Coefficient). Pearson's correlation matrix (distance d = 1-r >0 d: [0 2]) is a common way to represent the relationships between nodes in a network, where the correlation coefficient r measures the strength and direction of the linear relationship between two variables. The distance d is calculated as $d = 1 - r$, which transforms the correlation coefficient into a distance metric that can be used to analyze the network structure. In this representation, a high correlation (close to 1) corresponds to a small distance (close to 0), indicating that the nodes are closely related, while a low correlation (close to -1) corresponds to a large distance (close to 2), indicating that the nodes are not closely related.

It is a fully connected weighed network, which is very difficult to analyze and visualize. Connectivity can be reduced by considering its MST: if we find high correlation between two nodes, we can consider them as connected, otherwise not (correlation is a measure of similarity between two nodes: when the correlation is high, the nodes are similar as $A \equiv B$, $B \equiv C$, then $A \equiv C$ if A and C are also highly correlated). Doing this, we can find the backbone of the network, which is a tree structure that captures the most important relationships between the nodes. This transformation allows us to analyze and visualize the network more effectively, as it reduces the complexity while preserving the essential connections. However, it is important to note that this approach lead to the **loss of some information**, as it removes weaker connections that may still be relevant in certain contexts.

LIMITS: ALL cycles are removed: very drastic transformation

1.8.2 | Link Selection Rules

Global rules for link selection

We can also consider other global rules for link selection, which involve setting a threshold to determine which links to keep in the network. For example:

- consider the total distribution of link weights W and define a cutoff threshold C to keep only $W_{ij} > C$;
- define a parametric link distribution (eg Gaussian, Gamma) and keep only links so that $p(W_{ij}) < p_0$.

But this approach has some limitations, as it can lead to the loss of important information about the network structure:

- Global thresholding methods miss the local relevance of low link values (different weight distribution as a function of local node features such as their degree).

Local rules for link selection

Local methods analyze weights in node neighborhoods Link selection based on single-node null model (pdf)

Example (Serrano Boguna Vespignani PNAS07).

Node of degree K with positive weighted links W_{ij} (normalized to 1, can use absolute value if also negative) Hypothesis: weights are extracted from a random split of unit segment [0:1] in K parts (geometric distribution $G(K)$) Keep only links for which $G(W_{ij}) < p_0$ (global threshold but applied to local node-dependent distribution)

1.9 | Spectral Analysis of Networks

Through the analysis of the eigenvalues and eigenvectors of matrices associated with a network, such as the adjacency matrix or the Laplacian matrix, we can gain insights into the structural properties of the network. Spectral analysis can reveal important information about the connectivity, community structure, and dynamics of the network. For example, the largest eigenvalue of the adjacency matrix can indicate the presence of hubs in the network, while the second smallest eigenvalue of the Laplacian matrix can provide information about the connectivity and robustness of the network. Spectral methods are widely used in various applications, including clustering, dimensionality reduction, and network visualization.

1.9.1 | The Eigenvalue Spectrum

We can read the problem as an **eigenvalue problem**:

$$A \cdot \mathbf{x} = \lambda \cdot \mathbf{x},$$

where A is the adjacency matrix of the network, $\mathbf{x}$ is an eigenvector, and λ is the corresponding eigenvalue. The eigenvalues and eigenvectors can provide insights into the structure and properties of the network, such as identifying communities, detecting central nodes, and understanding the dynamics of processes on the network. Spectral analysis can also be used to identify important features of the network, such as its connectivity, robustness, and resilience to perturbations.

Example (Principal Component Analysis (PCA)). Principal Component Analysis (PCA) is a dimensionality reduction technique that can be applied to network data to identify the most important features or components that capture the variance in the data. By analyzing the eigenvalues and eigenvectors of the covariance matrix of the network data, PCA can help to identify patterns and relationships between nodes, as well as reduce the complexity of the data while preserving its essential structure. PCA can be particularly useful for visualizing high-dimensional network data and for identifying clusters or communities within the network.

We can also imply the **characteristic polynomial** of the adjacency matrix A, defined as

$$\Phi(\lambda) = \det(A - \lambda \mathbb{I}),$$

where I is the identity matrix. The roots of this polynomial are the eigenvalues of A, and analyzing them can provide further insights into the structural properties of the network.

1.9.2 | Powers of the Adjacency Matrix

Let us start by defining the powers of the adjacency matrix A. The n-th power of the adjacency matrix, denoted as A^n, can be computed using matrix multiplication. The entry $(A^n)_{ij}$ represents the number of paths of length n from node i to node j in the network:

$$B = A^2, \quad B_{ij} = \sum_k A_{ik} A_{kj}, \quad B_{ii} = \sum_k A_{ik} A_{ki}.$$

This is because each multiplication of the adjacency matrix counts the number of ways to traverse from one node to another through intermediate nodes. This multiplication tells us how many paths of length n exist between nodes i and j. For example, if $A^2_{ij} > 0$, it means there is at least one path of length 2 (since we are looking at the second power) connecting nodes i and j.

B_{ii} can infere on directed and undirected paths in the network, as it counts the number of paths that start and end at the same node i. In an **undirected network**, this would count the number of loops or cycles that involve node i, while in a **directed network**, it would count the number of directed paths that return to node i. This information can be useful for understanding the local structure and connectivity of the network around node i.

- The first non-null term ij of the k-th power of A indicates the length of the shortest path from i to j (without specifying the route(s)).
- The k-th power matrix counts the number of walks from one node to another.
- If an element $A^k_{ij} = 0$ for each k, there are no paths between the two nodes (up to $k_{\max} =$ #nodes).
- If the graph is directed, certain paths/walks may not exist.
- If no paths exist between two nodes in a symmetric graph, then the network is disconnected.

1.9.3 | Spectra of Specific Graphs

Cycles A *directed cycle* is a path that starts and ends at the same node, following the direction of the edges

$$(C_{ND})^N = \mathbb{I} \quad \Longrightarrow \quad \lambda_i^N = 1 \implies \lambda_i = e^{2\pi i k/N} \quad \text{for } k = 0, 1, \ldots, N-1,$$

showing that the eigenvalues of the adjacency matrix of a directed cycle graph are the N-th roots of unity. This means that the eigenvalues lie on the unit circle in the complex plane, and their distribution can provide insights into the structure and properties of the directed cycle graph. The presence of cycles in a network can have significant implications for its dynamics and stability, as they can lead to feedback loops and complex interactions between nodes. For example, if $N = 4$ we have:

$$C_{4D} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

[...]

Completely connected graphs These graphs have a simple structure, where every node is connected to every other node. The adjacency matrix of a completely connected graph with N nodes can be represented as:

all-ones matrix, rank 1, with eigenvalues N and 0 (with multiplicity $N-1$)

If we remove the self-loops (i.e., the diagonal entries), we get a traceless matrix with $N-1$ entries equal to 1 in each row, which can be represented as:

$$A = \begin{pmatrix} 0 & 1 & 1 & \cdots & 1 \\ 1 & 0 & 1 & \cdots & 1 \\ 1 & 1 & 0 & \cdots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \cdots & 0 \end{pmatrix},$$

which is traceless and has $N-1$ entries equal to 1 in each row (since each node is connected to $N-1$ other nodes). The diagonal entries are zero because there are no self-loops in the graph. This structure leads to a highly symmetric adjacency matrix, which has implications for its spectral properties and the overall connectivity of the network.

The eigenvalues of this adjacency matrix can be calculated, and they consist of one eigenvalue equal to $N-1$ (which corresponds to the eigenvector with all entries equal) and $N-1$ eigenvalues equal to -1 (which correspond to the eigenvectors that are orthogonal to the all-ones vector). This spectral property reflects the high level of connectivity in the completely connected graph, where each node is directly connected to every other node. The presence of a single large eigenvalue indicates that there is a strong overall connection in the network, while the negative eigenvalues indicate that there are no other significant structures or communities within the network.

Bipartite graphs

DAG

Note on Cycles and Network Sparsity

If a network has no cycles there is a single path between any two nodes (which is the shortest path necessarily): it is a powerful approximation for many real networks, which are often very sparse (e.g. with a small number of links as compared to the number of nodes $o(N^2)$). If a network is sufficiently sparse (e.g. with a small number of links as compared to the number of nodes o(N2)) it is very likely not to have cycles, or they can be considered negligible (e.g. in generating multiple paths between nodes): we can consider the network as a **tree**, or more generally as a DAG (if directed), without cycles.

This is an approximation used very often in theorems, in particular for random networks ensembles (see following lectures) In sparse networks cycles are rare, and long cycles even more.

Thus the presence of cycles is an indicator of a particular network design (that cannot be obtained by randomly connecting nodes, see following lectures on network entropy)

What happens to the spectrum of these matrices if we perturb slightly the network structure? For example, if we add or remove a few edges or loops (ciclicity, bipartition, all-ones...), how does this affect the eigenvalues and eigenvectors of the adjacency matrix? This question is important for understanding the stability and robustness of the network, as well as for analyzing the dynamics of processes that occur on the network. Perturbations can lead to changes in the spectral properties of the network, which can in turn affect its overall behavior and functionality. They can be studied by analyzing the sensitivity of the eigenvalues and eigenvectors to changes in the adjacency matrix, generating random matrices and analyzing their spectra, or by using perturbation theory to understand how small changes in the network structure can lead to changes in its spectral properties (like adding random links or random weights to the adjacency matrix). This analysis can provide insights into the resilience of the network to perturbations and its ability to maintain its functionality under changing conditions.

1.9.4 | Isomorphism and Co-spectrality

Two graphs are isomorphic if they are equal up to node permutation (transformation of similitude in matrix algebra)

$$A_1 = P\,A_2\,P^T, \quad \big(P\,P^T = \mathbb{I}\big)$$
$$|A_1| = |P\,A_2\,P^T| \implies |A_1| = |A_2|,$$

thus isomorphic graphs have the same spectrum, but the converse is not necessarily true: two non-isomorphic graphs can have the same spectrum (called **cospectral** graphs):

$$\text{Isomorphic} \implies \text{Cospectral},$$
$$\text{Non-Cospectral} \implies \text{Non-Isomorphic}.$$

This means that while the spectrum of a graph can provide important information about its structure and properties, it is not a complete invariant for graph isomorphism. Therefore, additional methods or invariants may be needed to determine whether two graphs are isomorphic or not, especially in cases where they are cospectral.

But co-spectral DOES NOT imply isomorphic! (for N> 4 knots) [Can one hear the shape of a drum? M. Kac Am Month. 73 (4), 1966]

Let us resume some of the key points about the spectrum of a graph:

- Eigenvalue spectrum: “Summary”, “modes” of a graph, tells us about the structure of the graph, but not all the information (e.g. it does not tell us about the specific arrangement of nodes and edges, or about the presence of certain substructures or motifs in the graph)

- Spectrum is independent of representation (invariant for node re-indexing)

- It is invariant for permutation operation: being a similarity transform (involving a matrix and its inverse) the eigenvalue problem is identical (related to the properties of the determinant of matrix products) problem is identical (related to the properties of the determinant of matrix products)

- There are few exact results and some numerical results for particular cases (more on trees, less on general graphs)

Co-spectrality

1.9.5 | Graph Distances

Once the eigenvalues have been sorted (with the addition of zeros for networks with different number of nodes, corresponding to isolated nodes that do not contribute to spectrum calculation)

$$\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_N,$$

$$\mu_1 \geq \mu_2 \geq \ldots \geq \mu_N, \mathrm{d}(G_1, G_2) = \sqrt{\sum_{i=1}^{N} (\lambda_i - \mu_i)^2},$$

we can define a spectral distance between two graphs G_1 and G_2 as the Euclidean distance between their sorted eigenvalue spectra. This distance provides a measure of how similar or different the two graphs are in terms of their spectral properties. A smaller distance indicates that the graphs have more similar spectral characteristics, while a larger distance indicates that they are more different. This spectral distance can be used for various applications, such as clustering graphs, comparing network structures, and analyzing the evolution of networks over time. However, it is important to note that this distance is not a complete invariant for graph isomorphism, as two non-isomorphic graphs can have the same spectrum (co-spectral graphs). Therefore, while the spectral distance can provide useful insights into the similarity of graphs, it should be used in conjunction with other methods for a more comprehensive analysis of graph structure and properties.

Similar Graphs and Their Spectra

To define similar graphs we have to define a distance between graphs, which can be done using the spectral distance defined above. Similar graphs are those that have a small spectral distance, indicating that their eigenvalue spectra are close to each other. This means that the structural properties of the graphs are similar, as reflected in their spectra. Similar graphs may have similar connectivity patterns, community structures, or other structural features that contribute to their spectral similarity. However, it is important to note that similar graphs may not necessarily be isomorphic, as they can have different arrangements of nodes and edges while still having similar spectral properties. Therefore, while the spectral distance can provide insights into the similarity of graphs, it should be used in conjunction with other methods for a more comprehensive analysis of graph structure and properties.

There seems to be a strong correlation between spectral distance and graph edit distance. Thus proximity of spectra reflects graphs with similar structure (even if exceptions exist)

TODO: insert graphs

Graph edit distance Graph edit distance is a generalization of edit distance between strings of symbols, which is used in bioinformatics or in text analysis to identify similarity in different strings (protein or DNA sequences, words or periods). In symbol sequences, we can have elementary operations like symbol substitution, symbol insertion or symbol removal.

The set of operations that connect one string to the other can be represented in a graph table, with a path representing the editing steps. Different scores can be defined to identify the best matching between two strings (and thus between two graphs), for example

- Longest common subsequence (subgraph)
- Global alignment scores based on different values of penalties associated to mismatches, insertions, deletions

Example (DNA sequences). For proteins and DNA sequences there is a natural node ordering, thus edit distance is simpler to calculate (substitution/insertion/deletion along the sequences). For networks there is no natural node ordering, thus also the starting point of node-to-node mapping is nontrivial. Possible approaches (open problems):

- Spectral approaches to matrices (eigenvectors components and node similarities)
- Similarity between node embeddings

2 Random Graph Ensembles

In this chapter, we will explore the concept of random graph ensembles, which are fundamental in understanding the properties and behaviors of complex networks. We will start with the simplest model, the Erdős-Rényi random graph, and then move on to more complex models such as the Stochastic Block Model (SBM). We will also discuss the thermodynamic limit, degree distribution, phase transitions, and other properties of these random graph ensembles.

2.1 | Erdős-Rényi Random Graphs

We are going to start with the simplest model of random graphs, which is the Erdős-Rényi (E-R) model. This model serves as a fundamental null model for understanding the properties of complex networks.

Definition 2.1 (Erdős-Rényi Random Graphs)**.** Networks whose interactions (links) between nodes are chosen randomly, without a priori correlations or rules. In statistical mechanics terms, it is comparable to a "perfect gas" with molecules (nodes) experiencing random "sparse" impacts (links).

It can be seen as a **null model** for complex networks, as it represents a system with no structure or correlations.

If we expect to find a certain property in a real network, we can compare it to the expected value of that property in an E-R random graph. If the observed value significantly deviates from the expected value, we can infer that there is some underlying structure or correlation in the real network that is not captured by the E-R model. It can also be used to confront different systems, such as comparing the structure of a social network to that of a biological network, to identify unique features and commonalities.

If we make the statistical analogy with a gas, the E-R model represents a perfect gas, where the nodes are like molecules that interact randomly without any specific rules or correlations. The edges, which are generated randomly, represent some average quantities of the system, not the distance between nodes or the strength of interactions. The E-R model is a useful starting point for understanding the properties of complex networks, but it is important to note that real-world networks often exhibit more complex structures and correlations that are not captured by this simple model.

2.2 | Statistical Ensembles of Networks

If we imagine to take a picture of our ideal gas system, we would see a random distribution of molecules (nodes) and their interactions (links, diluted interactions, since we are speaking of ideal gas). The E-R model captures this randomness and serves as a baseline for understanding the properties of complex networks. In the next instant, the picture would change, and we would see a different random distribution of molecules and interactions, but the overall properties of the system would remain the same, as they are determined by the average quantities rather than the specific configuration of nodes and links. We will see that these analogies with ensambles will be very useful to understand the properties of complex networks, and also the E-R model will mantain some properties constant even if we change the specific configuration of nodes and links, which is a key feature of random graph ensembles.

We are going to use **ensembles of networks** with fixed parameters: same macroscopic state (e.g. link density, degree pdf) different microstates (single-node properties), and we will see that the properties of the ensemble are determined by the average quantities rather than the specific configuration of nodes and links. The definition of the ensamble will suggest a construction rule to generate the microstates of the ensemble and to compute the probability of generating a specific microstate.

> **Remark.** *The number of ALL symmetric networks (microstates) with N nodes (whatever the links) is:*
>
> $$2^{\frac{N(N-1)}{2}}, \quad \sum_{i=1}^{N} \binom{N}{i} = 2^N.$$
>
> *This means that the number of possible configurations of links between N nodes is exponential in the number of nodes, which highlights the complexity of the space of possible networks and the importance of using ensembles to understand their properties.*

The most common ensembles are the microcanonical ensemble, where we fix the number of links, and the canonical ensemble, where we fix the probability of having a link between two nodes. Let us introduce these two ensembles in more detail.

2.2.1 | Microcanonical Ensemble $G_{N,M}$

Definition 2.2 (Microcanonical Ensemble)**.** An ensemble of networks with N nodes, representing a fixed system size, and M links, representing the system energy. The links are randomly selected from the possible $N(N-1)/2$ for a symmetric network or $N(N-1)$ for a direct graph.

The idea behind a microcanonical ensemble is that we are looking at all possible configurations of a network with a fixed number of nodes and links. Each configuration (or microstate) is equally likely, and the properties of the ensemble are determined by the average quantities rather than the specific configuration of nodes and links.

Looking at the properties of the microcanonical ensemble, we can see that it is a very simple model, but it can still capture some important properties:

- The N nodes represent the system size, which is fixed.
- The M links represent the system energy, which is also fixed.

- The links are randomly selected from the possible $N(N-1)/2$ for a symmetric network or $N(N-1)$ for a directed graph, which means that there is no specific structure or correlation in the network.

So we have

$$M_{max} = \frac{N(N-1)}{2}, \quad \#G_{N,M} = \binom{M_{max}}{M},$$

such that for a fixed number of nodes N and links M, the number of possible configurations (or microstates) of the network is given by the binomial coefficient $\binom{M_{max}}{M}$, which counts the number of ways to choose M links from the total possible M_{max} links. This highlights the combinatorial nature of the microcanonical ensemble and the importance of using statistical methods to understand its properties.

Remark (Construction Rule). *The construction rule of one instance of the ensemble is:*

1. *Take N nodes.*
2. *Repeat M times: choose 2 nodes randomly and connect them, checking for loops or existing links.*

The probability of generating "bad" links, such as loops and multilinks, is about $1/N$. The effect of this selection becomes negligible for large N.

The probability of generating a loop is $1/N^2$, and the probability of generating a multilink is M/N^2. In order to connect two times the same nodes, the probability is $1/N^2$. For large N, i.e. in the thermodynamic limit, these probabilities become negligible, and the microcanonical ensemble can be effectively generated by randomly selecting M links from the possible $N(N-1)/2$ links without worrying about loops or multilinks.

Given the number of nodes N and links M, the properties of the microcanonical ensemble are determined as well as the whole ensamble itself. Indeed with only those two parameters, we can build every possible configuration of the network, and we can compute the average properties of the ensemble, such as the degree distribution, clustering coefficient, and other network metrics.

2.2.2 | Canonical Ensemble $G_{N,P}$

Often it is more convenient to **fix the probability** of having a link between two nodes, rather than the number of links. This leads us to the canonical ensemble.

Definition 2.3 (Canonical Ensemble)**.** An ensemble of networks with N nodes (fixed system size) and a fixed probability P to have a link between them, representing a fixed average energy or temperature of the system. The links are generated randomly with probability P from the possible $N(N-1)/2$ for a symmetric network or $N(N-1)$ for a directed graph.

This means that in the canonical ensemble, given the nodes and the probability P, we can generate a random graph by connecting each pair of nodes with a link with probability P. The properties of the ensemble are determined by the average quantities:

$$P = \langle E \rangle \propto T, \quad \langle l \rangle = P\frac{N(N-1)}{2}.$$

Binomial Link Distribution

Given N and P, we do not have a fixed number of networks belonging to the canonical ensemble. But we can inquire something more general: ALL graphs with N nodes belong to $G_{N,P}$ with a different probability depending on P.

The probability of having a graph with m links and N nodes is governed by the binomial distribution:

$$P(m) = \binom{N(N-1)/2}{m} P^m (1-P)^{N(N-1)/2-m}.$$

The average grows as

$$\langle m \rangle = P \cdot \binom{N}{2} = P\frac{N(N-1)}{2},$$

with a variance of

$$\sigma_m^2 = P(1-P) \cdot \binom{N}{2} = P(1-P)\frac{N(N-1)}{2}.$$

Remark (Construction Rule). *The probability of having a graph with m links and N nodes is governed by the binomial distribution. To construct an undirected graph:*

- *Take $N(N-1)/2$ values A_{ij} uniformly distributed between 0 and 1.*
- *If $A_{ij} > 1-P$ then $A_{ij} = 1$, else $A_{ij} = 0$.*
- *Apply symmetrization using logic "OR": $A \vee A^T$.*

If the network is sufficiently large, the probability of having a graph with m links and N nodes is governed by the binomial distribution, which can be approximated by a Poisson distribution in the limit of large N and small P with $P \cdot N$ kept constant. The average number of links in the canonical ensemble is given by $\langle l \rangle = P\frac{N(N-1)}{2}$, which grows as the square of the number of nodes for fixed P. This highlights the fact that in the canonical ensemble, the number of links can vary widely depending on the specific configuration of nodes and links, but the average properties of the ensemble are determined by the probability P and the number of nodes N.

2.2.3 | Equivalence of Ensembles

For $N \gg 1$, the canonical link distribution tends to a Dirac's delta function. All instances of the canonical ensemble tend to have a number of links equal to the average link number, thus recovering the microcanonical property.

[...]

2.2.4 | Degree Probability Density Function (pdf)

The probability of having a node with k links is the probability of randomly choosing k links among $N-1$:

$$p(k) = \binom{N-1}{k} P^k (1-P)^{N-1-k},$$

Balls and Holes

If we consider the problem of distributing k links (balls-couples) among $N-1$ possible connections (holes), we can see that the degree distribution follows a **binomial distribution**, which can be approximated by a Poisson distribution in the limit of large N and small P with $P \cdot N$ kept constant. This analogy helps us understand the degree distribution in random graphs and how it arises from the random process of connecting nodes with links.

The degree distribution in the canonical ensemble is given by the binomial distribution, which can be approximated by a Poisson distribution in the limit of large N and small P with $P \cdot N$ kept constant

$$p(k) = \binom{N-1}{k} P^k (1-P)^{N-1-k}$$

The average node degree $\langle k \rangle$ is given by $P(N-1)$, which grows linearly with the number of nodes for fixed P:

$$\langle k \rangle = \frac{\langle \#\text{links} \rangle}{N} = \frac{2P\frac{N(N-1)}{2}}{N} = P(N-1) \approx PN,$$

while the average link density is

$$p\frac{N(N-1)}{2}.$$

This highlights the fact that in the canonical ensemble, the degree distribution can vary widely depending on the specific configuration of nodes and links, but the average properties of the ensemble are determined by the probability P and the number of nodes N.

Remark (Bernoulli to Poisson to Delta). *In the thermodynamic limit ($N \to \infty$, $P \to 0$) with $P \cdot N = \lambda$ kept constant, the distribution becomes a Poisson distribution:*

$$p(k) \approx \frac{\lambda^k}{k!} e^{-\lambda}$$

In this limit, a random Bernoulli graph tends to a regular graph with equal connectivity for all nodes, even if node proximity is not regular as in a lattice. The Erdős-Rényi model tends to an infinite-dimensional regular lattice.

$$P \cdot N = \lambda = \textit{constant}(P \to 0, N \to \infty),$$
$$p(k) = \frac{\lambda^k}{k!} e^{-\lambda}, \quad \langle k \rangle = \lambda, \quad \sigma = \sqrt{\lambda},$$
$$\lim_{\lambda \to \infty} p(k) = N(\lambda, \sqrt{\lambda}) \to \delta(x - \lambda).$$

[...]

2.3 | Phase Transitions and the Giant Component

Considering the probability of having a property in an ensemble with N nodes

$$P = P(N),$$

depending on the trend we get behaviors that lead to have certain properties valid for almost all graphs or almost none. This is the idea of a phase transition, where we have a critical point that separates two different phases of the system. In the context of random graphs, one of the most important properties is the emergence of a giant component, which is a connected component that contains a finite fraction of the nodes in the graph. The emergence of a giant component is a phase transition that occurs at a critical value of the average connectivity λ.

2.3.1 | Average Connectivity

In E-R networks, the *threshold for transitions depends on the network size N and the connection probability p.* We use the average connectivity λ as the control parameter:

$$\langle k \rangle \approx pN = \lambda$$

2.3.2 | The Giant Component

Definition 2.4 (Giant Component Emergence).

- For $\lambda < 1$, there is no giant component.
- For $\lambda \geq 1$, there is a giant component that, for increasing λ, tends to include all the nodes.

Remark (Transcendental Equation). *If we define u as the probability of a node not belonging to the Giant Component, the average field equation yields the transcendental equation:*

$$u = e^{\lambda(u-1)}$$

The order parameter S, which is the fraction of nodes belonging to the giant component, is $S = 1 - u$.

... GC order parameter...

One can find similarities with the **percolation model**: If we have a lattice of nodes and we randomly occupy the links with probability p, we can see that there is a critical value of p above which a giant component emerges, which is similar to the phase transition in random graphs. The percolation model can be seen as a special case of the random graph model, where the nodes are arranged in a lattice and the links are occupied with a certain probability.

Adding one link at a time creates larger and larger clusters, which is known as the "**avalanche phenomenon**". This is similar to the process of percolation, where adding one link can lead to the emergence of a giant component that contains a finite fraction of the nodes in the graph: after a critical point, the network becomes traversable, connected. The phase transition in random graphs can be understood as a percolation process, where the control parameter is the average connectivity λ.

2.4 | Network Properties in Random Graphs

We can expect that in a random graph, the properties of the network will be determined by the average quantities rather than the specific configuration of nodes and links. This means that we can use statistical methods to understand the properties of random graphs and how they arise from the random process of connecting nodes with links.

2.4.1 | Diameter and Average Path Length

Assuming a regular graph ($N \gg 1$) and a sparse network (regular tree, no cycles, so that from a node there is a unique path to any other node and a very small probability of comeing back to the starting node) with

$$\#l \sim O(N) = o(N^2),$$

we can define the **diameter** of the graph as the longest shortest path between any two nodes.

Starting from a node, one can reach $\langle k \rangle$ nodes in one step, $\langle k \rangle^2$ nodes in two steps, and so on. After d steps, one can reach $\langle k \rangle^d$ nodes. Therefore, the diameter of the graph can be estimated by solving the equation:

$$\langle k \rangle^d \sim N.$$

In a random graph, the diameter grows logarithmically with the number of nodes, which means that even for large networks, the distance between nodes remains relatively small. This is a key feature of random graphs and is often referred to as the "small-world" property.

Let us give some insight into the solution of that equation. We can take the logarithm of both sides to get:

$$\dots$$

A similar result holds for the **average path length**

$$\langle l(N) \rangle \propto \ln N.$$

As the network size grows, the distance between nodes grows slowly, which for a D-lattice, translates in the diameter growing as $N^{1/D}$, which is much faster than logarithmic growth.

Random networks are **traversable**, meaning that there is a path between any two nodes, and the distance between nodes grows logarithmically with the number of nodes. This is a key feature, a trivial property of random graphs and is often referred to as the "**small-world**" property, which has important implications for the dynamics and functionality of complex networks.

2.4.2 | Degree Correlations

Let us study the relationship between the degrees of neighboring nodes in a random graph. In an E-R random graph, there is no correlation between the degrees of neighboring nodes, which means that the degree of a node does not depend on the degree of its neighbors. This is because the links are generated randomly, and there is no specific structure or correlation in the network. If the nodes are correlated, we would expect to see a relationship between the degrees of neighboring nodes, such as a positive correlation (where high-degree nodes tend to be connected to other high-degree nodes) or a negative correlation (where high-degree nodes tend to be connected to low-degree nodes). However, in an E-R random graph, there is no such correlation, and the degree of a node is independent of the degree of its neighbors.

There is an **absence of correlations** between node features. Specifically, there is ALMOST none for finite-size graphs and none in the thermodynamic limit ($N \to \infty$ and $\#l = O(N)$). For undirected graphs:

$$p(k_1, k_2) \approx p(k_1) \cdot p(k_2).$$

In-Degree and Out-Degree Correlations

Similarly, in an ER directed network, there is no correlation between in-degree and out-degree: $p(k_{IN}, k_{OUT}) = p(k_{IN}) \cdot p(k_{OUT})$. This means that the in-degree of a node does not depend on its out-degree, and vice versa. This lack of correlation is a key feature of random graphs and distinguishes them from more structured networks, where there may be correlations between the degrees of neighboring nodes.

2.5 | Stochastic Block Model (SBM)

Another important model of random graphs is the Stochastic Block Model (SBM), which is a more complex null model that can modelize a nontrivial community structure of the network. It can be obtained by combining diagonal E-R blocks, possibly of different sizes and link densities, with non-diagonal blocks of random links. The diagonal blocks represent the communities, where nodes within the same community are more likely to be connected, while the non-diagonal blocks represent the connections between different communities, which can be random or have a different link density.

Definition 2.5 (Stochastic Block Model)**.** A more complex null model that can modelize a non-trivial community structure of the network. It can be obtained by combining diagonal E-R blocks, possibly of different sizes and link densities, with non-diagonal blocks of random links.

The main algorithm backbone is the same to generate symmetric or directed E-R networks of specific size and link density. The non-diagonal blocks connecting communities of different size (e.g. with N and M nodes) will be rectangular ($N \times M$).

What makes the SBM more complex than the E-R model is that it can capture the community structure of real-world networks, where nodes are organized into groups or communities that have a higher density of connections within the group than between groups. This makes the SBM a more realistic model for understanding the properties of complex networks, as many real-world networks exhibit community structure.

We can play with the structure of the blocks to generate different types of networks, such as assortative networks (where nodes within the same community are more likely to be connected) or disassortative networks (where nodes within the same community are less likely to be connected). We can model networks with a core-periphery structure, where there is a dense core of nodes that are highly connected to each other and a periphery of nodes that are less connected. We can also model networks with a hierarchical structure, where there are multiple levels of communities nested within each other. The SBM is a powerful tool for understanding the properties of complex networks and can be used to generate synthetic networks that mimic the structure of real-world networks.

3 Spectral Analysis of Random Matrices

Spectral analysis applied to adjacency matrices (and matrices derived from them, such as the Laplacian) provides a deep perspective on the macroscopic and topological properties of networks. By studying the eigenvalues and eigenvectors of these matrices, we can deduce crucial information about paths, the presence of cycles, network robustness, and the structure of hidden communities.

In the study of random networks, attention is primarily focused on three mathematical observables:

- **Eigenvalue distribution**: to understand how the global energy or information of the network is distributed.
- **Maximum eigenvalue** (λ_{max}): strictly related to connectivity and diffusion limits on the network.
- **Nearest Neighbor Spacing Distribution** (NNSD): useful for identifying local correlations between the states of the network.

These spectral properties can be analyzed using tools from random matrix theory, which provides a framework for understanding the statistical properties of eigenvalues in large random matrices. By studying the spectral properties of random graphs, we can gain insights into the structure and dynamics of complex networks, and how they differ from more structured networks with community structure or other correlations. Our study cases will be:

- Random symmetric matrix, with entries drawn from a Gaussian distribution, which can be used to model the adjacency matrix of an E-R random graph.
- E-R matrix, which is a specific case of a random symmetric matrix where the entries are binary (0 or 1) and represent the presence or absence of links between nodes in an E-R random graph.
- Hi-C matrix, which is a specific case of a random symmetric matrix that represents the contact frequencies between different regions of the genome in a Hi-C experiment. The spectral properties of Hi-C matrices can provide insights into the 3D organization of the genome and how it relates to gene regulation and other cellular processes.

3.1 | Gaussian Orthogonal Ensemble (GOE)

Definition 3.1 (Gaussian Orthogonal Ensemble)**.** The Gaussian Orthogonal Ensemble (GOE) is one of the fundamental models in random matrix theory. It represents an ensemble of symmetric matrices where the independent elements (i.e., those on the diagonal and in the upper triangle) are random variables drawn from normal (Gaussian) distributions.

This model is widely studied in physics and mathematics because its eigenvalue distribution has an explicit, symmetric form and can be calculated very efficiently. The term "orthogonal" derives from the fact that the statistical properties of these matrices remain invariant when subjected to orthogonal transformations.

3.1.1 | Wigner Semicircle Law

For a random symmetric $N \times N$ matrix whose elements are sampled from a Gaussian distribution $\mathcal{N}(0, \sigma)$, the eigenvalue spectrum as $N \to \infty$ follows the famous **Wigner semicircle law**.

img/wigner_semicircle.png

Figure 3.1: Eigenvalue spectrum of a GOE random matrix, showing the Wigner semicircle distribution.

This means that the real eigenvalues tend to cluster together, forming a semicircle shape centered at the origin. The radius of this semicircle depends on the number of nodes N and the standard deviation σ of the original distribution:

$$\lambda \in [-2\sigma\sqrt{N}, 2\sigma\sqrt{N}].$$

The probability density function of the eigenvalues is defined as:

$$\rho(\lambda) = \frac{\sqrt{4N\sigma^2 - \lambda^2}}{2\pi N\sigma^2}.$$

3.1.2 | Eigenvector Components

In the GOE model, not only do the eigenvalues follow precise rules, but the components of the normalized eigenvectors are also distributed according to a Gaussian probability density. From a network science perspective, this implies that all eigenvectors are essentially delocalized and indistinguishable. They are therefore "uninformative" for finding hidden structures, as the Gaussian noise covers any pattern.

From the central limit theorem, if we have set of randomly distributed variables, their sum will tend to a Gaussian distribution, regardless of the original distribution of the variables. In the case of random graphs, the eigenvectors are linear combinations of the entries of the adjacency matrix, which are randomly distributed. Therefore, the components of the eigenvectors will also be randomly distributed and will tend to a Gaussian distribution due to the central limit theorem. This means that all eigenvectors will be equally distributed and will not provide any meaningful information about the structure of the graph, which is a key feature of random graphs.

img/gaussian-eigenvectors.png

Figure 3.2: Components of eigenvectors in a GOE random matrix, showing the Gaussian distribution.

3.2 | Spectrum of Erdős-Rényi (E-R) Networks

When we analyze the adjacency matrix associated with a connected, symmetric Erdős-Rényi (E-R) random network, we notice that the Wigner semicircle law approximately holds for the majority of the eigenvalues (the so-called "bulk"). However, the discrete nature of the connections (presence or absence of a link) brings out some specific topological characteristics.

Remark (E-R Spectrum and the Isolated Eigenvalue). *The spectrum of an E-R network consists of the typical semicircular bulk (representing random topological noise) plus a single isolated principal eigenvalue (outlier), which distinctly detaches from the semicircle and scales linearly with N.*

- *The standard deviation associated with the matrix is $\sigma = \sqrt{N \cdot P(1-P)}$, where P is the connection probability.*
- *The isolated maximum eigenvalue can be approximated as $\lambda_{max} \approx P \cdot N + 1$.*

The presence of this large eigenvalue is a key feature of E-R random graphs and distinguishes them from more structured networks, where there may be multiple large eigenvalues corresponding to different communities or other structures in the graph.

Furthermore note that:

$$\mu \neq 0, \quad P = \frac{1}{N(N-1)} \sum_{ij} A_{ij},$$

which means that the presence of a non-zero mean in the distribution of the matrix elements (i.e., a non-zero average degree in the graph) leads to the emergence of an isolated eigenvalue that scales with N: the average degree introduces a global bias in the connectivity of the graph, which is reflected in the spectrum of its adjacency matrix.

img/E-R-spectrum.png

Figure 3.3: Eigenvalue spectrum of an E-R random graph, showing the semicircular bulk and the isolated maximum eigenvalue.

As the system size N increases, the radius σ of the semicircle grows proportionally to $\sqrt{N}$, while the maximum eigenvalue λ_{max} grows much faster, proportionally to N. Consequently, the ratio between the radius and the maximum eigenvalue tends to zero. Macroscopically, the eigenvalue spectrum tends to converge towards that of a regular graph where all nodes possess the exact average connectivity $\langle k \rangle$.

Limits of the Maximum Eigenvalue λ_{max}

By the Perron-Frobenius Theorem applied to non-negative matrices, the maximum eigenvalue of a network is unique and strictly tied to the node degrees. It is mathematically bounded by the average degree $\langle k \rangle$ and the maximum degree k_{max} of the graph:

$$\langle k \rangle \leq \lambda_{max} \leq k_{max}$$

This parameter is of fundamental importance in the study of dynamical processes on networks. For example, it governs the spreading rate of a virus in a contact network or defines the stability of synchronization among coupled oscillators.

img/directed_GOE_dist.png

Figure 3.4: Eigenvalue distribution of a directed GOE random matrix in the complex plane: the eigenvalues are uniformly distributed within a circle of radius $r = \sigma\sqrt{N}$.

3.2.1 | Directed Networks and the Circular Law

When moving to directed (oriented) $N \times N$ random matrices, symmetry is lost. As a result, the eigenvalues are no longer confined to the real number axis. Instead, they form a uniform circular distribution in the complex plane, known as the "Circular Law," with a radius of $r = \sigma\sqrt{N}$. Even in a directed E-R network, we find this uniform distribution on the complex plane, almost always accompanied by the usual purely real and isolated eigenvalue.

To better understand the spectrum of random networks, it is useful to consider the deterministic baseline of regular graphs. In a **k-regular graph**, every node is connected to exactly k neighbors.

- **Principal Eigenvector:** Every k-regular graph admits an eigenvector where all components are equal, $\mathbf{v} = (1, 1, \ldots, 1)^T$. This highly delocalized vector corresponds exactly to the maximum eigenvalue $\lambda_{max} = k$.

- **Completely Connected Network:** In a fully connected network (or clique), every node is connected to all other nodes, making it a regular graph with $k = N - 1$. Its spectrum is highly degenerate: it features a single principal eigenvalue $\lambda_{max} = N - 1$, while the remaining $N - 1$ eigenvalues are all equal to -1. The sum of all eigenvalues (the trace of the adjacency matrix) remains exactly 0, reflecting the absence of self-loops.

Remark (E-R Spectrum, Wigner Limit, and Disconnected Components). *In an Erdős-Rényi graph, the expected average degree is $\langle k \rangle = NP$. Because the network behaves macroscopically like a regular graph with this average degree, we observe the isolated principal eigenvalue at $\lambda_{max} \approx NP + 1$. Meanwhile, the rest of the eigenvalues λ tend toward the Wigner semicircle distribution, representing the random fluctuations around this average connectivity.*

Furthermore, network connectivity directly impacts the zero eigenvalues: if the graph is disconnected (i.e., fragmented into multiple isolated subgraphs), the spectrum of its Laplacian matrix will feature an eigenvalue $\lambda = 0$ with an algebraic multiplicity equal to the exact number of disconnected components.

img/directed_er_dist.png

Figure 3.5: Eigenvalue distribution of a directed E-R random graph in the complex plane, showing the circular law and the isolated maximum eigenvalue: the eigenvalues are uniformly distributed within a circle of radius $r = \sigma\sqrt{N}$, while the maximum eigenvalue is located at $\lambda_{max} \approx NP + 1$. The presence of the isolated eigenvalue indicates the average connectivity of the graph, while the circular distribution reflects the random nature of the connections.

3.2.2 | Structured Networks: Toeplitz and Circulant Matrices

While random matrix theory deals with pure topological noise, many real-world networks exhibit strong spatial or deterministic structures that profoundly alter their spectra.

- **Toeplitz Matrices:** In a Toeplitz adjacency matrix, all diagonal bands have constant values: $a_{ij} = f(i - j)$. Physically, this implies that the connection weights between nodes depend solely on their one-dimensional distance. This is a common model for networks sorted along a linear chain, such as polymer backbones or sequential protein structures.
- **Circulant Matrices:** A circulant matrix is a special, highly symmetric case of a Toeplitz matrix where each row is a cyclic shift (modulo N) of the preceding row. Topologically, this corresponds to a network with periodic boundary conditions, such as a closed ring lattice. Because of this translational symmetry, the eigenvectors of circulant matrices are independent of the specific matrix elements and can be elegantly solved using the Discrete Fourier Transform.

3.3 | Nearest Neighbor Spacing Distribution (NNSD)

In addition to the global shape of the spectrum, it is very useful to study the differences between adjacent eigenvalues. Considering the ordered sequence of eigenvalues $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_N$, we define the spacing between two consecutive eigenvalues as $s_n = \lambda_{n+1} - \lambda_n$.

Definition 3.2 (NNSD)**.** The Nearest Neighbor Spacing Distribution (NNSD) analyzes the statistical distribution of the spacing ratios $r_n = \frac{s_{n+1}}{s_n}$. For a matrix belonging to the GOE, this distribution explicitly follows the formula:

$$P(r) = \frac{27}{8} \frac{(r + r^2)}{(1 + r + r^2)^{5/2}}$$

This distribution highlights the phenomenon of *level repulsion*: unlike purely random numbers that can cluster freely, the eigenvalues of a random matrix tend to "repel" each other, making it highly improbable to find two identical or very close eigenvalues. This universality property is faithfully preserved even in random E-R graphs.

img/nnsd-goe-er.png

Figure 3.6: Nearest Neighbor Spacing Distribution (NNSD) for a GOE random matrix and an E-R graph: both follow the same distribution, confirming the universality of level repulsion.

3.3.1 | Biological Application: Hi-C Matrices

Hi-C is a sophisticated experimental technique used in molecular biology to estimate the spatial proximity of DNA regions within the cell nucleus (3D structure of the genome). The genome is divided into fragments called "bins" (with typical resolutions of 1 Mb or 100 kb). The experiment generates a large symmetric adjacency matrix, where each element A_{ij} represents the observed frequency of physical contact between bin i and bin j.

img/hi-c-experimental_pipeline.png

img/hi-c-matrix.png

Figure 3.7: Hi-C experimental pipeline and resulting contact matrix: the Hi-C technique generates a large symmetric matrix where each element represents the frequency of physical contact between different regions of the genome. The spectral analysis of this matrix can reveal important insights into the 3D organization of the genome and its functional implications.

Hi-C Spectrum and Noise Filtering (Essential Matrix)

Applying spectral analysis to a real Hi-C matrix, crucial differences emerge compared to purely random mathematical models:

- **Eigenvalues**: A central "bulk" is observed that follows GOE rules (representing the pure background noise of the experiment) and a pronounced tail of isolated eigenvalues that contain the true structural biological "signal".
- **Eigenvectors**: Unlike the purely Gaussian components of the GOE, the eigenvectors corresponding to the signal eigenvalues show heavy tails. This indicates that they carry differentiated and highly localized biological information, useful for identifying topologically associating domains (TADs).

img/hi-c-spectrum_noise.png

img/hi-c-eigenvec_dist.png

Figure 3.8: Hi-C spectrum and eigenvector distribution: the spectrum of a real Hi-C matrix shows a clear separation between a central bulk (noise) and isolated eigenvalues (signal). The eigenvectors corresponding to the signal eigenvalues exhibit heavy-tailed distributions, indicating that they carry localized biological information.

By exploiting this clear separation between noise (the bulk) and structural information (the outliers), it is possible to clean the data by reconstructing an **Essential Hi-C Matrix**. Similar to Principal Component Analysis (PCA), the original network is projected only onto the first n^* principal eigenvectors that carry the real signal:

$$A_{ij}^{ess} = \sum_{n=1}^{n^*} \lambda_n a_n^{(i)} a_n^{(j)}$$

img/essential_hi-c.png

Figure 3.9: The essential Hi-C matrix in comparison to the original one:...

Finally we can compare the NNSD for the Hi-C matrix with the GOE distribution. This tells us about only the short-range correlations in eigenvalues, thus capturing the universality present in a wide range of complex networks.

img/nnsd-hi-c_goe_er.png

Figure 3.10: Comparison...

3.4 | Free Probability Theory and the Laplacian

3.4.1 | Generalization of the Semicircle

The strength of the semicircle law lies in its robustness: it applies not only to matrices with Gaussian elements but also to matrices where the weights are drawn from arbitrary distributions, provided they are independent and have finite variance. This concept is the basis of the Central Limit Theorem for matrices. Furthermore, the spectrum resulting from the sum of two independent ("free") random matrices can be elegantly calculated using the so-called "free convolution" of their individual spectra.

3.4.2 | Covariance Matrices and the Spectrum of the Laplacian

Covariance matrices generated from random data do not follow the semicircle law, but rather the renowned **Marchenko-Pastur** eigenvalue distribution. Since the Laplacian matrix L of a random directed graph can be factorized as the product of the incidence matrix I and its transpose ($L = I^T I$), its algebraic structure is similar to that of a covariance matrix. Consequently, the eigenvalue spectrum of the Laplacian of an E-R graph is directly modeled by the Marchenko-Pastur distribution.

`img/marcenko-pastur-dist.png`

Figure 3.11: ooooooooooo

Remark (Participation Ratio). *To rigorously quantify whether the components of an eigenvector are homogeneous (delocalized) or concentrated on a few specific nodes (localized), the* ***Participation Ratio (PR)*** *is calculated. Given a normalized eigenvector where* $\sum v_i^2 = 1$*, the PR is defined as:*

$$PR = \sum_{i=1}^{N} v_i^4$$

The conceptual limits of this metric are:

- $PR \to \frac{1}{N}$*: The eigenvector is perfectly delocalized (e.g., a completely uniform network where all nodes have equal weight).*
- $PR \to 1$*: The eigenvector is totally localized (all information is concentrated on a single node).*

In E-R networks, the eigenvectors of the Laplacian exhibit "heavy" tails. However, even though they are not Gaussian, the PR values remain statistically homogeneous across the various eigenvectors, confirming the absence of dominant hubs and the substantial topological indistinguishability of nodes in the Erdős-Rényi model.

4 Network Properties and Centrality Measures

To fully characterize network properties, we evaluate measures that can be broadly divided into global and local categories.

Global measures evaluate the network as a whole entity, providing insights into its overall structure and behavior. These measures are crucial for understanding the macroscopic properties of the network and how it functions as a complete system. Examples of global features include:

- The total number of nodes and edges, as well as the overall link density, which indicates how interconnected the network is.

- Structural metrics such as the diameter, girth, genus, and the total clustering coefficient, which provide insights into the network's topology and the presence of cycles and clusters.

- Spectral properties, which involve the analysis of the network's eigenvalues and eigenvectors, offering insights into its stability, robustness, and dynamic behavior.

- Assortativity and mixing patterns, which describe the tendency of nodes to connect with similar or dissimilar nodes based on certain attributes (e.g., degree, type).

- The presence and structure of network communities and motifs, which reveal the modular organization and recurring patterns within the network.

Local measures, conversely, focus on characteristics at the single node or edge level, or on the relationships between pairs of nodes or edges. These include:

- Node and edge centrality measures, which assess the importance or influence of individual nodes or edges within the network.

- One-dimensional node or link feature distributions, such as degree distributions, which provide insights into the heterogeneity of connectivity.

- The relation between parameter distributions, often modeled as N-dimensional probability density functions (e.g., the joint distribution of degree and clustering coefficient), which can reveal correlations and dependencies between different local properties.

Node and Link Centrality Measures

Centrality measures are parameters that can be associated with every single node or link, or specific groups of them. They provide vital information regarding the relevance and importance of specific network elements and are typically characteristic of the type of network being considered.

These measures are particularly useful for:

- Ranking and stratifying nodes and links based on their structural importance.
- Identifying peculiar elements, acting as outliers within the related statistical distributions of the network.

4.1 | Degree Centrality

Degree, or connectivity, is a fundamental **local parameter**. It represents the total number of incoming or outgoing links connected to a specific node. In terms of linear algebra, for a directed network, this corresponds to the row or column sum of the adjacency matrix.

Definition 4.1 (Degree)**.** Let $K_{IN}(i)$ represent all the links entering the i-th node, and $K_{OUT}(i)$ represent all the links exiting the i-th node. They are defined as:

$$K_{IN} = \sum_j a_{ij}, \quad K_{OUT} = \sum_i a_{ij}$$

For symmetric (undirected) networks, the adjacency matrix is equal to its transpose ($A = A^T$), meaning that $K_{IN} = K_{OUT}$. The underlying premise of degree centrality is straightforward: a node that is very connected is considered very important. Typical examples of this dynamic include website hyperlinks, academic citations, and friendship connections.

4.1.1 | Degree Distribution

The **distribution of connectivity** across all nodes provides important information regarding the broader properties of the network. In particular, examining the heterogeneity of K values, represented by the probability density function $p(k)$, helps characterize specific types of networks. Common types of degree distributions include:

- **Poisson pdf:** Characteristic of the Erdős-Rényi (E-R) ensemble, often used as a baseline null model for random networks.
- **Scale-free pdf:** Characteristic of networks grown via preferential attachment, such as the Barabási-Albert (B-A) model, which leads to a power-law distribution of degrees.
- **Fat-tailed pdf:** Arises from several other complex stochastic processes.

Example (Human Protein Interaction Network). An excellent biological example of a non-trivial structure is the human Protein-Protein Interaction (PPI) network (e.g., PathwayCommons). This is a hierarchical network demonstrating a highly heterogeneous degree distribution that behaves very differently from a random E-R graph. The network is massive, consisting of more than 11,000 nodes and over 420,000 links , and is heavily characterized by a fat-tailed $p(k)$ distribution.

Figure 4.1: The human PPI network is a hierarchical network with a highly heterogeneous degree distribution, characterized by a fat-tailed $p(k)$ distribution. It consists of over 11,000 nodes and more than 420,000 links, demonstrating a structure that is very different from a random E-R graph.

img/human_ppi_network.png

4.1.2 | Multivariate Analysis of Degree

In general, it is highly informative to consider the **joint distribution of couples of centrality measures**, often visualized using 2D histograms, density plots, or XY scatterplots. While some parameter pairs (e.g., *betweenness centrality* vs. *degree*) follow expected patterns defined by standard null models, non-trivial patterns frequently emerge in real networks, allowing us to identify interesting elements such as specialized nodes or links.

Erdős-Rényi (E-R) ensembles serve as the simplest null models to establish a baseline for these comparisons, as they are defined by a single parameter (the link density λ) and do not exhibit any correlations between node degrees.

Two standard 2D joint distributions analyzed are:

- K_{IN} vs K_{OUT} (in a directed graph): in real networks (non E-R), we often observe non-trivial patterns that can indicate specific functional roles of nodes (e.g., "hubs" with high outdegree but low indegree).
- K vs K_{NN} (the average degree of a node's first neighbors): real networks often show significant correlations in this relationship, which can help identify specialized nodes (e.g., "effector" nodes with low degree but high-degree neighbors, or "master" nodes with high degree but low-degree neighbors).

Definition 4.2 (Degree of Nearest Neighbors)**.** The sum of the degrees of all first neighbors of node i (i.e., nodes directly connected to the considered node) is:

$$K_{NN}(i) = \sum_{i \sim j} K_j.$$

The average degree of the nearest neighbors can be expressed as:

$$\langle K_{NN}(i) \rangle = \frac{1}{N_{NN(i)}} \sum_{i \sim j} K_j,$$

while, if we define the vector of degrees as $\vec{K} = (K_1, K_2, \ldots, K_N)$, we can express its nearest neighbor degree as a matrix operation:

$$\vec{K}_{NN} = \sum_j A_{ij} \cdot K_j = A \cdot \vec{K} \oslash \vec{N},$$

where $\oslash$ denotes the Hadamard (element-wise) division.

Example (Joint Distributions in Null Models vs Real Networks). In an E-R directed random graph (our null model), there is no statistical relationship between indegree (K_{IN}) and outdegree (K_{OUT}), nor is there a relationship between a node's degree (K) and the degree of its neighbors (K_{NN}). The scatterplots appear uniform and uncorrelated.

img/scatterplot-densityplot-ER.png

Figure 4.2: In the left scatterplot of an E-R directed random graph, there is no statistical relationship between the NN degree (K_{NN}) and the node's degree (K), resulting in a uniform distribution. In the right one, the same happens with K_{IN} and K_{OUT}.

Conversely, biological networks like the Yeast PPI (Protein-Protein Interaction) network exhibit a non-trivial K vs K_{NN} correlation. This specific pattern helps identify specialized biological roles, such as “effector” and “master” nodes. If we perform a link reshuffling on this network (randomizing connections while preserving degrees), this observed pattern is completely destroyed, proving it is a product of evolutionary design rather than random chance.

img/yeast_ppi.png

Figure 4.3: The Yeast PPI network (Maslov Sneppen Science 2002) shows a non-trivial correlation between K_0 and K_1, as well as for K and K_{NN}, which helps identify specialized biological roles. This pattern is destroyed by link reshuffling, indicating it is a product of evolutionary design. In the center we can see a schemaic view of the network, where we can identify “effector” nodes (low degree but high-degree neighbors) and “master” nodes (high degree but low-degree neighbors).

4.1.3 | Weighted Networks and Node Strength

For weighted networks where links possess a specific **weight** ($W_{ij} > 0$), the concept of degree is extended to “strength”. It should be noted that the degree of a node in a weighted network is still defined as the number of links connected to it, regardless of their weights. However, the strength of a node provides a more nuanced measure of its connectivity by accounting for the weights of the links.

Definition 4.3 (Node Strength)**.** The strength of a node is defined as the sum of the weights of all links connected to it:

$$S_i = \sum_j W_{ij}.$$

Analyzing the relationship between topological degree (K) and strength (S) reveals how weights are distributed across the network. Two common patterns are observed:

- **Linear Relationship ($S_i \propto K_i$):** This indicates a random null model where weights are assigned independently of the node’s connectivity. In this case, the strength of a node is directly proportional to its degree, suggesting that weights are distributed uniformly across the network.
- **Superlinear Relationship ($S_i \propto K_i^{\beta}$ with $\beta > 1$):** This indicates a non-random association where highly connected nodes also attract disproportionately heavier weights. A prime example is the US airport weighted network, where weights (traffic) are a function of distances and broader topological metrics.

Remark (Weighted and Topological Networks). *If we have a weighted network matrix W, we can always extract its underlying topological adjacency matrix A by applying an element-wise logical function: $A = W > 0$. Whenever analyzing a weighted network, it is best practice to study the original and the "binarized" (topological) network in parallel to isolate the specific role the weights play: if the weighted and topological analyses yield similar results, then the weights are not adding significant information beyond the topology. Conversely, if the results differ significantly, it indicates that the weights are crucial for understanding the network's structure and function.*

4.1.4 | Leafs and Core Networks

Nodes with a degree of exactly $K = 1$ are known as pendant nodes, or "*leafs*". Conceptually, the number of leafs can be viewed as the network's "*surface*", while the total number of nodes represents its "*volume*".

We can structurally reduce a network using the **Pendant Node Removal Algorithm**, which iteratively deletes all nodes with $K = 1$. This iterative pruning will result in one of two outcomes:

1. **Nothing remains:** This implies the original network was a pure tree (containing no cycles), where all nodes are leafs except for a few cutpoints. In this case, the network's structure is entirely defined by its degree distribution.

2. **A sub-network remains:** We are left with a "core network" without any leafs. In this core, every node has $K \geq 2$, indicating the presence of structural cycles. The properties of this core network are not solely determined by the degree distribution, as the presence of cycles introduces additional structural complexity.

4.2 | Closeness Centrality

We are going to define the **closeness centrality** of a node, which is a foundamental measure of the node's importance based on its average distance to all other nodes in the network.

Definition 4.4 (Closeness Centrality)**.** Closeness is a measure linked to the **mean shortest path distance** (P_{ij}) from one node to all others (*reachability*). The core idea is that a node less distant from all others is more important.

$$Cl_i = \frac{N-1}{\sum_j P_{ij}} = \frac{1}{\langle P_i \rangle}$$

Closeness characterizes the *optimal spread capacity of a node* within a few steps—whether it is spreading information, a commodity, or energy. It is a particularly robust measure in systems where there is a probability of **signal corruption** or information loss at each step along a path. In such cases, the shortest path is not necessarily the most efficient route for spreading, as it may be more susceptible to corruption. Instead, a node with a higher closeness centrality can reach other nodes through multiple paths, increasing the likelihood of successful transmission despite potential losses.

Remark (Notes on Closeness).

- Cl_i *typically exhibits a very small range of values across a network. This is because, in most complex networks, the overall diameter scales logarithmically with the number of nodes,* $\log(N)$*.*

- *Special care must be taken with disconnected components. If a network is fragmented, the distance to an unreachable node is mathematically infinite, which will break the standard closeness formulation.*

- *Notably, while closeness is calculated for a single local node, its value depends entirely on the global properties of the network.*

Example (Closeness and Epidemic Spreading). Closeness centrality has powerful real-world applications in epidemiology. By combining time series infection data (the onset day of an infection in different regions) with network data representing average human mobility (tracked via mobile phones), researchers can forecast local outbreak timelines. Nodes with higher closeness centrality in the mobility network directly correlate with earlier infection onset days. This relationship is supported by significant Pearson and Spearman correlation values, demonstrating that closeness centrality is a strong predictor of epidemic spread dynamics.

img/closeness_epidemic-spreading.png

Figure 4.4: Closeness centrality of nodes in a mobility network correlates with the onset day of an epidemic outbreak: nodes with higher closeness centrality experience earlier infection onset, as shown by the significant Pearson and Spearman correlation values.

	Pearson r	r **p-value**	**Spearman** ρ	ρ **p-value**
Geo Dist	0.276	1×10^{-3}	0.267	1×10^{-2}
Net Dist	0.420	5×10^{-5}	0.516	$< 1 \times 10^{-6}$
Closeness	-0.520	$< 1 \times 10^{-6}$	-0.555	$< 1 \times 10^{-6}$
1/pop	0.578	$< 1 \times 10^{-6}$	0.548	$< 1 \times 10^{-6}$

Table 4.1: Correlation between infection outbreak day and different distance/centrality metrics.

4.3 | Betweenness Centrality

In the study of complex networks, while degree centrality measures the local connectivity of a node, **Betweenness Centrality** (BC) quantifies its role in intermediation and traffic flow. It is based on the assumption that information (or physical traffic) travels predominantly along the shortest paths between pairs of nodes. A node with high betweenness acts as a critical bridge or a cutpoint, controlling the flow between different, otherwise disconnected, regions of the network.

Definition 4.5 (Betweenness Centrality)**.** Let $G = (V, E)$ be a graph. The Betweenness Centrality BC_i of a node i (or equivalently, an edge) is defined as the fraction of all shortest paths in the network that pass through it:

$$BC_i = \sum_{m \neq i \neq n} \frac{\#P_{mn}(i)}{\#P_{mn}}$$

where $\#P_{mn}$ is the total number of shortest paths from node m to node n, and $\#P_{mn}(i)$ is the number of those shortest paths that actually pass through node i.

In practical applications, computing the exact shortest paths requires specific algorithms depending on the network properties:

- **Dijkstra's Algorithm:** Used for networks with only positive link weights. It computes all shortest paths from a single source node in $\mathcal{O}(N^2)$ or $\mathcal{O}(E \log V)$ time.
- **Floyd-Warshall Algorithm:** Computes the shortest paths for all node pairs simultaneously in $\mathcal{O}(N^3)$ time. It allows for signed links and its matrix-based formulation makes it highly suitable for distributed computing optimization across multiple cores.

Brokers (or bridges). It is worth noting that a node might have a very low degree k but a disproportionately high betweenness centrality. Such nodes act as *brokers* or *bridges* between dense clusters. In a typical scatterplot of BC versus k, these nodes are easily identifiable as outliers that deviate from the expected positive correlation.

Limitations. The fundamental hypothesis of BC is that interactions occur *only* along the shortest paths. In many real-world scenarios, this is an oversimplification: longer paths might be preferred to avoid traffic jams or bottlenecks, requiring the introduction and analysis of further topological weights.

img/bridge_BC.png

Figure 4.5: Betweenness Centrality (BC) quantifies the importance of a node based on the fraction of shortest paths that pass through it.

4.3.1 | Normalized Betweenness

The absolute value of Betweenness Centrality scales with the network size N, because larger networks naturally possess more paths (and it is related to the average path length). To compare the betweenness of nodes belonging to different networks, a proper normalization is required.

The theoretical maximum value for betweenness centrality is achieved by the central node in a **star network** topology. In this configuration, all peripheral nodes (which are $N-1$) must route their traffic through the central hub. The number of possible pairs among these peripheral nodes is given by the binomial coefficient:

$$BC_{MAX} = \binom{N-1}{2} = \frac{(N-1)(N-2)}{2}$$

We can thus define the **normalized** (or absolute) **betweenness centrality**, BC_{ABS}, by scaling the raw value with respect to BC_{MAX}:

$$BC_{ABS} = \frac{BC}{BC_{MAX}}$$

Scaling properties. The minimum possible value of BC corresponds to a leaf node (a node connected to only one other node). The paths going from one node to all others (and vice versa) scale as $2N$, meaning $BC_{min} \sim \mathcal{O}(N)$. In contrast, for a star network hub, $BC_{max} \sim \mathcal{O}(N^2)$. Consequently, the normalized measure BC_{ABS} ranges from $\mathcal{O}(1/N)$ up to $\mathcal{O}(1)$. When a network exhibits very **high normalized betweenness values**, it structurally resembles a tree: a very sparse graph with a distinct absence of cycles that could generate alternative paths.

4.3.2 | Betweenness and Global Efficiency

Betweenness centrality is intimately connected to the concept of network efficiency. The **Global Efficiency** (E) of a network measures its overall capability to transfer information and is defined as *the mean of the inverse of the lengths of all shortest paths* (often referred to as a "harmonic composition"):

$$\epsilon_{ij} = \frac{1}{P_{ij}}$$

$$E = \langle \epsilon_{ij} \rangle = \frac{1}{N(N-1)} \sum_{i \neq j} \frac{1}{P_{ij}}$$

It is a powerful tool for quantifying the overall performance of a network in terms of **information transfer** (total transferability). We can measure the importance of a single node (or link) i by evaluating the *local efficiency variation* after a structural perturbation. Specifically, we remove the i-th node/link and compute the relative drop in Global Efficiency.

Example (Immune System Network Analysis). The concepts of betweenness and perturbative efficiency can be elegantly applied to biological systems, such as the human Immune System. We can model the system using nodes to represent different cells (e.g., B lymphocytes, T cells, neutrophils - about 19 types) and links to represent mediators (e.g., cytokines, chemokines, hormones - around 100 kinds).

The system is initially described by two directed bipartite graphs:

1. Cells producing mediators (directed edges from cells to mediators).
2. Mediators targeting cells (directed edges from mediators to cells).

By contracting this bipartite structure, we obtain a directed, weighted cell-cell network where the edge weights correspond to the number of distinct mediators enabling communication between two cells. Because the same mediators often connect multiple different cells, the network is densely connected and the weights are widely distributed.

`img/immune_sys_example.png`

Figure 4.6: A sub-network illustrating the directed, weighted cell-cell graph obtained from the contraction of the bipartite cell-mediator network. Nodes represent distinct immune system cells (e.g., B lymphocytes, dendritic cells, granulocytes, and mast cells). The directed edges represent communication pathways, with weights (e_{ij}) explicitly corresponding to the number of distinct chemical mediators (such as specific interleukins, TNF-α, or TGF-β) that facilitate the targeted interaction.

Perturbative Analysis. To understand the relevance of specific mediators, we study the perturbative effect of removing a single mediator from the system. Since a single mediator acts on multiple cells, its removal alters several edge weights simultaneously. The impact is quantified by the variation in the Global Efficiency of the projected network:

$$\Delta E = \frac{|E - E'|}{E} = \frac{|E(G) - E(G')|}{E}$$

where G' is the network after the mediator removal.

Biological Interpretation. Ranking the mediators based on their ΔE (which acts as a measure of centrality) yields profound biological insights. Top-ranking mediators are primarily associated with the *innate immune system* (which provides a non-specific response to external antigens), rather than the adaptive one. This suggests that network centrality encodes evolutionary history: the most central elements are also the most ancient ones in the evolutionary timeline of the organism.

Similar Multi-partite Examples. This specific modeling approach—where the exchange of distinct "mediators" creates weighted connections between macroscopic entities—is ubiquitous in complex systems:

- **Economics:** Countries exchanging different quantities of specific goods (oil, gold, wheat).
- **Finance:** Markets interacting via the exchange of various currencies (dollars, pounds, yen).
- **Biochemistry:** Different metabolic reactions connected by the utilization of the same ubiquitous compounds (e.g., ATP, H_2O, CO_2) or atomic elements (like the carbon cycle or electron exchange in redox reactions).

4.4 | Eigenvalue Centrality

A fundamental shift in the definition of centrality occurs when we posit that a node is "important" if it is connected to other "important" nodes. This recursive concept is the foundation of Eigenvalue Centrality (EC), originally introduced in sociology and mathematics (see Katz 1953, Bonacich 1987, and for a comprehensive review, Newman's "Networks: An Introduction").

Definition 4.6 (Eigenvalue Centrality)**.** Let G be a network described by an adjacency matrix A. The **Eigenvalue Centrality** EC_i of a node i is defined such that it is proportional to the sum of the centralities of its neighbors. Mathematically, by introducing a proportionality constant $\frac{1}{\lambda}$, this is expressed as:

$$EC_i = \frac{1}{\lambda} \sum_j A_{ij} \cdot EC_j$$

In vector notation, denoting the vector of centralities as $\vec{x}$, this recursive relation naturally translates into a classic eigenvalue problem:

$$A \cdot \vec{x} = \lambda \cdot \vec{x}$$

4.4.1 | Solving the Eigenvalue Problem

The solution for the Eigenvalue Centrality is given by the components of the eigenvector relative to the maximum eigenvalue, λ_{MAX}:

$$\lambda_{MAX} : EC_i = x_i^{\lambda_{MAX}}$$

By the Perron-Frobenius theorem (for non-negative matrices), the eigenvector associated with the largest eigenvalue λ_{MAX} is guaranteed to have all strictly positive components ($x_i > 0$), making it the most physically meaningful solution for a centrality measure.

Note on other eigenvectors: Technically, other eigenvectors of the adjacency matrix could be used to assign values to nodes. However, positivity for all components is no longer guaranteed for eigenvectors associated with $\lambda \neq \lambda_{MAX}$. Despite this, these alternate eigenvectors can still carry structural insights, and one could potentially sort nodes based on these signed values to identify specific topological features or network bipartitions.

4.4.2 | Eigenvalue Centrality in Directed Networks

When dealing with directed graphs, the adjacency matrix A is generally asymmetric. Consequently, we have two distinct options for computing Eigenvalue Centrality, depending on the flow of information:

- **Right eigenvector:** This measures the centrality of nodes based on being **pointed to** by other nodes (receiving incoming links).
- **Left eigenvector:** This measures the centrality of nodes based on **pointing to** other nodes (generating outgoing links).

Pathologies and Limitations. Directed networks introduce specific structural problems for Eigenvalue Centrality:

1. If a node j has an in-degree $K_{IN} = 0$ (meaning $A_{ij} = 0$ for all i), its eigenvalue centrality is strictly zero: $EC_j = 0$.
2. Because the measure is recursive, if a node is pointed to *only* by nodes that have $EC = 0$, its own centrality will also be forced to zero.
3. In general, directed networks suffer from issues related to separated components (e.g., source or sink components). This heavily impacts the overall eigenvector spectrum. In matrix terms, this corresponds to block matrices with absorbing states (sinks), which can localize the centrality on a very small subset of the network.

4.5 | Hubs and Authorities (HITS Algorithm)

To resolve some of the issues of directed networks, we can distinguish between two mutually reinforcing roles a node can play. A classic example is a network of academic paper citations:

- **Authorities ($\vec{x}$):** Nodes containing highly relevant information. They are characterized by being pointed to by many hub papers.
- **Hubs ($\vec{y}$):** Nodes that act as extensive directories or review articles, pointing to many papers with relevant information (authorities).

Importantly, the same node i possesses both an authority score x_i and a "hubness" score y_i.

Definition 4.7 (Hubs and Authorities Scores)**.** In a directed network described by an adjacency matrix A, each node i is assigned an **authority score** x_i and a **hub score** y_i. These quantities are defined by a mutually recursive relationship, where a good authority is pointed to by good hubs, and a good hub points to good authorities. Using proportionality constants α and β, the system is formalized as:

$$x_i = \alpha \sum_j A_{ij} \cdot y_j \implies \vec{x} = \alpha A \vec{y}$$

$$y_i = \beta \sum_j A_{ji} \cdot x_j \implies \vec{y} = \beta A^T \vec{x}$$

4.5.1 | Combining the Equations

The two systems of equations can be decoupled by substitution, leading to two separate eigenvalue problems. Let $\lambda = \frac{1}{\alpha\beta}$:

1. Substituting $\vec{y}$ into the first equation:

$$\vec{x} = \alpha\beta A A^T \vec{x} \implies A A^T \vec{x} = \lambda \vec{x}$$

2. Substituting $\vec{x}$ into the second equation:

$$\vec{y} = \alpha\beta A^T A \vec{y} \implies A^T A \vec{y} = \lambda \vec{y}$$

Since the matrix $A^T A$ (like AA^T) is by definition symmetric, the two distinct problems share the **same eigenvalue spectrum** (real eigenvalues), although they yield **different eigenvectors**. The eigenvector of AA^T provides the authority scores, while the eigenvector of $A^T A$ provides the hub scores.

Network Interpretation. These matrices correspond to the structural symmetrization approaches used for directed graphs:

- AA^T represents the **cocitation network** (where a link indicates two nodes are cited by the same source).
- $A^T A$ represents the **bibliographic coupling network** (where a link indicates two nodes cite the same target).

4.5.2 | Notes on the HITS Algorithm

This coupled approach forms the basis of the **HITS (Hyperlink-Induced Topic Search) algorithm**, developed by J. M. Kleinberg (*Authoritative sources in a hyperlinked environment, J. ACM 46, 604-632, 1999*; also discussed in Newman's book).

From a computational standpoint, one does not need to solve both eigenvalue problems from scratch. Given the equations:

$$AA^T\vec{x} = \lambda\vec{x}$$

$$A^TA\vec{y} = \lambda\vec{y}$$

It can be easily seen that $\vec{y} = A^T\vec{x}$ acts as the required eigenvector. Thus, we only need to compute the eigenvalue problem for AA^T to find the authority vector $\vec{x}$, and then derive the hub vector $\vec{y}$ through a simple matrix multiplication.

Final Note: Although the HITS algorithm is mathematically and conceptually very elegant, it is actually not much used in modern broad-scale applications, as it has not (yet) found overwhelmingly interesting practical uses compared to other ranking algorithms like PageRank.

4.6 | Network Backbone and Salient Links

While centrality measures like Betweenness Centrality (BC) evaluate the importance of a node or a link based on the total number of shortest paths crossing it, this approach can sometimes obscure the fundamental skeleton of a network. To extract the true "backbone" of a complex graph, we introduce the concept of **Salient Links**.

4.6.1 | Definition of Link Salience

Given a graph, instead of looking at all individual shortest paths simultaneously, we consider the *shortest-path spanning tree* T_k rooted at a specific node k. This tree represents the most efficient pathways to reach all other nodes in the network starting from k.

The **Salience** of a link (i, j), denoted as S_{ij}, is defined as the fraction of these shortest-path trees that contain the link (i, j). Mathematically, it is the average over all possible root nodes:

$$S_{ij} = \frac{1}{N} \sum_{k=1}^{N} T_{ij}(k)$$

where $T_{ij}(k) = 1$ if the link (i, j) belongs to the spanning tree T_k, and 0 otherwise. Similarly, the global salience of the network can be expressed as an average of the trees:

$$S = \langle T \rangle = \frac{1}{N} \sum_{k=1}^{N} T_k$$

Crucial Note: Link Salience is **not** equivalent to Edge Betweenness Centrality. While BC counts the absolute multiplicity of shortest paths, Salience treats the entire shortest-path tree of a node as a single coherent structure, measuring how indispensable a link is across all global broadcast trees.

4.6.2 | Distributions: Continuous vs. Bimodal

In many real-world complex systems (e.g., transportation networks like air traffic, biological food webs, or global trade networks), standard link-level centrality measures such as Link Weight (W) and Link Betweenness (BC) exhibit a **continuous, often broad or scale-free distribution**.

img/link_centrality_distribution.png

Figure 4.7: Typical continuous distributions of link-level centrality measures (e.g., Link Weight, Link Betweenness) in complex networks. These distributions often show heavy tails, indicating a wide range of link importance.

However, Link Salience (S) behaves fundamentally differently. In many weighted graphs, the salience of links spontaneously stratifies, exhibiting a **bimodal distribution** $p(s)$. The distribution is sharply peaked near $S \approx 0$ (links that are rarely part of any spanning tree) and $S \approx 1$ (links that are almost always necessary).

img/salience_distribution.png

Figure 4.8: Bimodal distribution of link salience in weighted graphs, showing spontaneous stratification and the emergence of a network backbone.

This bimodal nature reveals a "spontaneous stratification" of the network topology: it naturally filters the graph into a highly salient **network backbone** (the critical infrastructure of the network) and a set of redundant, non-salient peripheral connections.

4.6.3 | Topological Differences between BC and Salience

The conceptual difference between Betweenness Centrality and Salience becomes starkly apparent when analyzing homogeneous topologies, such as a planar graph with random connections between neighboring nodes.

img/planar_graph_BC_LS.png

Figure 4.9: A planar graph with random connections between neighboring nodes, serving as a test case to compare Betweenness Centrality and Link Salience. In such a homogeneous topology, BC tends to create a core-periphery structure, while Salience maintains a more uniform distribution of importance across links.

- **Betweenness Centrality:** Tends to artificially induce a *core-periphery differentiation*. Links near the topological center of the planar graph will inevitably intercept more shortest paths simply due to geometry, accumulating high BC values and creating a centralized "bottleneck."

- **Link Salience:** Maintains a *homogeneous distribution*. Because it evaluates links based on their presence in spanning trees rather than raw path counting, it does not falsely elevate the center, accurately reflecting the homogeneous nature of the underlying planar grid.

4.6.4 | Salient Links in Dynamical Processes

The structural backbone identified by salient links plays a critical role in dynamical processes, particularly in spreading and transport phenomena.

Consider a repeated random walk on a weighted network, where the passage of the walker through a node equates to an “infection” event. If we trace and “sign” the specific paths through which nodes become infected—effectively creating a random walk with memory (a *non-Markovian process*)—we observe a striking correlation with the network’s structural salience.

The expected number of passages through a given link during this dynamical spreading process is directly proportional to its salience:

$$\#\ \text{passages} \propto S_{link}$$

`img/salience_dynamics_correlation.png`

Figure 4.10: Correlation between link salience and the number of passages in a random walk with memory (non-Markovian process). Links with higher salience are traversed more frequently, indicating their critical role in the dynamics of spreading processes.

This demonstrates that the salient backbone is not just a static topological artifact; it provides the primary deterministic channels for information, disease, or traffic flow in complex systems.

4.7 | Comparing Centrality Measures on Toy Models

To build a solid intuition regarding how different centrality measures behave, it is highly instructive to compute and compare them on simple, canonical network topologies (toy models). By observing how Degree (K), Betweenness (BC), Closeness (CL), and Eigenvector Centrality (EC) fluctuate across these extreme structural cases, we can better understand their specific biases and applications.

4.7.1 | Symmetric Topologies: Complete and Cycle Graphs

In perfectly symmetric graphs, topological equivalence forces many centrality measures to flatten out, highlighting the baseline behavior of the metrics.

img/complete_graph_centrality.png

Figure 4.11: In a complete graph, all nodes are structurally equivalent, resulting in identical Degree, Closeness, and Eigenvector Centrality values across the network. Betweenness Centrality is zero for all nodes, as no node serves as a bridge on any shortest path.

- **Complete Graph (Clique):** In a fully connected network of N nodes, every node is connected to every other node. Consequently, all nodes share the exact same degree ($K = N-1$), Closeness, and Eigenvector centrality. Crucially, the Betweenness Centrality is exactly zero ($BC = 0$) for all nodes. Since every pair of nodes is connected by a direct edge (a path of length 1), no node ever acts as an intermediary or “bridge” on a shortest path.

img/cycle_graph_centrality.png

Figure 4.12: In a cycle graph (ring), all nodes have the same degree and identical centrality measures due to the graph's symmetry. This illustrates that local metrics cannot differentiate nodes in translationally invariant networks.

- **Cycle Graph (Ring):** Similar to the complete graph, the cycle graph exhibits perfect symmetry (periodic boundary conditions). Every node has $K = 2$. Because no node occupies a structurally privileged position, BC, CL, and EC are identical across the entire network. This demonstrates that local metrics cannot differentiate nodes in translationally invariant networks.

4.7.2 | Spatial Gradients: The Path Graph

Breaking the periodic boundary conditions of a cycle yields a path graph (a simple line of nodes), which immediately introduces a spatial gradient into the centrality measures.

img/path_centrality.png

Figure 4.13: In a path graph, nodes at the extremities have low centrality measures, while those near the center exhibit higher values. This illustrates how Betweenness and Closeness Centrality capture "topological interiority" in a linear structure.

In a path graph, nodes at the extremities (leaves) have a degree of 1 and $BC = 0$, as they cannot mediate any paths. As we move toward the geometric center of the line, both Betweenness and Closeness Centrality strictly increase, peaking at the central node(s). This topology clearly visualizes how BC and CL act as measures of "topological interiority," capturing how close a node is to the topological center of mass of the system.

4.7.3 | Structural Holes and Brokers: The Dumbbell Graphs

The most critical topological examples are those that decouple Degree Centrality from Betweenness Centrality, illustrating the concept of "brokers" or "structural holes."

`img/dumbbell_graph.png`

Figure 4.14: The dumbbell graph consists of two dense clusters connected by a single bridging node. This structure highlights the concept of "brokers" or "structural holes."

- **The Dumbbell (Single Cutpoint):** Consider two dense clusters (blocks/cliques) connected by a single bridging node. The nodes inside the clusters have high degree and high Eigenvector Centrality (because they are connected to many other well-connected nodes). However, the central connecting node (the cutpoint, e.g., Node 6 in the slide) has a relatively *low degree* ($K = 2$) but the *absolute maximum Betweenness Centrality*. This happens because 100% of the shortest paths traveling from one block to the other must bottleneck through this specific node. It is the ultimate "broker".

`img/dumbbell_redundant_cutpoint.png`

Figure 4.15: By adding alternative paths between the two blocks, we can create a "redundant cutpoint" scenario. This illustrates how Betweenness Centrality can be highly sensitive to network redundancy, as the central node's BC can drop to zero if it is bypassed by a shorter or equivalent route.

- **Redundant Cutpoint:** The sensitivity of Betweenness Centrality is highlighted if we slightly perturb the previous graph by adding alternative paths between the two blocks. If the cutpoint is bypassed by a shorter or equivalent route, its BC immediately crashes to zero. This shows that BC assumes routing strictly follows the absolute shortest paths, making it highly sensitive to network redundancy.

`img/tree_graph_centrality.png`

Figure 4.16: Even if the cutpoint is not bypassed, the presence of alternative paths can significantly reduce its Betweenness Centrality. This illustrates how BC can be diluted by network redundancy, even when the original bottleneck node remains a critical connector.

4.7.4 | Complex Community Structures

Real-world networks rarely look like perfect dumbbells, but they often exhibit a "community structure" or "core-periphery" organization that behaves similarly.

img/community_structure.png

Figure 4.17: A generic graph with a two-blocks community structure, illustrating how nodes at the boundaries can act as brokers.

In a 2-block network (characterized by many internal links within communities and very few cross-links between them), we see a macroscopic manifestation of the dumbbell effect. Nodes deep inside a community will boast high Degree and Eigenvector Centrality. Conversely, the few nodes positioned at the boundaries—those handling the sparse cross-links—will act as local bridges. Just like the cutpoint, these boundary nodes will exhibit a Betweenness Centrality that is disproportionately high compared to their degree. Identifying these nodes is crucial for understanding how to disrupt or facilitate communication between distinct communities in a complex system.

4.8 | Clustering Coefficient and Null Models

A fundamental property of many real-world networks is the **transitivity of connectivity**, commonly summarized by the social adage: "the friends of my friends are also my friends." The **Clustering Coefficient** is the structural metric designed to quantify this tendency of nodes to cluster together and form tightly knit groups.

Definition 4.8 (Local Clustering Coefficient)**.** For a given node i with degree k_i, let e_i be the actual number of links connecting its first neighbors (adjacent nodes) to each other. The local clustering coefficient C_i is defined as the ratio between e_i and the maximum possible number of links that could exist between those neighbors:

$$C_i = \frac{e_i}{\max(e_i)} = \frac{2e_i}{k_i(k_i-1)}$$

Geometrically, if two neighbors of node i are also connected to each other, these three nodes form a **triangle**. Therefore, C_i essentially measures the local density of triangles centered at node i. The value of C_i ranges from 0 (a purely star-like neighborhood with no interconnected neighbors) to 1 (a fully connected clique where all neighbors interact with each other).

img/clustering_coefficient.png

Figure 4.18: Illustration of the local clustering coefficient C_i for a node i. The numerator counts the actual number of links (e_i) between the neighbors of i, while the denominator counts the maximum possible number of links between those neighbors, which is given by $\frac{k_i(k_i-1)}{2}$.

An alternative, more formal way to express this transitivity is by counting cycles:

$$C_i = \frac{\#C_3}{\#C_2}$$

where $\#C_3$ represents the number of triangles (cycles of order 3) passing through i, and $\#C_2$ represents the number of connected triples centered at i (paths of length 2, or cycles of order 2).

To characterize the entire network, we compute the **Global Clustering Coefficient** C, which is simply the average of the local coefficients over all nodes in the network: $C = \langle C_i \rangle$.

4.8.1 | Comparison with the Erdős–Rényi Null Model

To determine if a network exhibits meaningful clustering, its global clustering coefficient C must be compared against an appropriate null model, typically an Erdős–Rényi (E-R) random graph with the same number of nodes and links, or a random configuration model preserving the exact degree sequence.

Based on this comparison, real networks generally fall into two broad structural categories:

- **Social Networks:** Typically exhibit $C \gg C_{random}$. This exceptionally high clustering is driven by **triadic closure**, a sociological mechanism where individuals sharing a mutual acquaintance have a high probability of becoming acquainted themselves.

- **Technological Networks:** Systems like the Internet or power grids often show $C \leq C_{random}$. These networks are typically designed for efficient distribution over large distances, naturally favoring a **tree-like structure** that avoids redundant, costly triangular loops.

4.8.2 | Applications: Functional Brain Networks

A prominent application of the clustering coefficient is in neuroscience, specifically in the topological analysis of functional brain networks derived from time-series data of brain activity (such as functional NMR).

The standard analytical pipeline involves several steps:

1. **Data Acquisition:** Time series are extracted across various spatial scales, ranging from high-resolution brain voxels ($\sim 10^5$) to broader, macroscopic brain regions ($\sim 10^2$).

2. **Network Construction:** A functional network is built by assessing the co-activation between regions. This is quantified using correlation-like measures (e.g., lagged Pearson's correlation, mutual information, or Granger causality). The result is a symmetric, fully connected, weighted correlation matrix.

3. **Filtering and Discretization:** To extract the underlying functional topology, the continuous correlation matrix is often binarized into a unweighted 0/1 network by applying a threshold C_{THRS}. Only functional links with a correlation $C_{ij} > C_{THRS}$ are retained.

Thresholding Effects and Pathological Synchronization. The choice of the threshold significantly impacts the resulting network topology. If a very high threshold is applied (e.g., 0.95), the remaining strong correlations tend to mathematically mimic an *equivalence relation*. In such relations, transitivity holds strictly (if region A is highly correlated to B, and B to C, then A is highly correlated to C). Consequently, correlation-based networks subjected to a high global threshold naturally exhibit an abundance of triangles, leading to characteristically high clustering coefficients.

Tracking region synchronization through the clustering coefficient has proven to be a robust clinical indicator. For instance, certain neurological pathologies, such as **epilepsy**, are structurally characterized by abnormally high functional brain synchronization. In these contexts, anomalous spikes in the clustering coefficient serve as a reliable biomarker.

Relevant Literature: The correlation between structural brain features and various centrality/clustering measures is extensively explored in literature (see *Zanghieri et al., Brain Sci 2021*; *Latora, arXiv 2010*; and *Testa, NeuroImage 2018*).

4.8.3 | The Erdős-Rényi Ensemble as a Null Model

To properly interpret the centrality and clustering values measured in real-world networks, we must compare them against a baseline where no underlying organizational principle exists. The **Erdős-Rényi (ER) random graph ensemble** serves as this fundamental null model. In an ER graph, nodes are statistically "indistinguishable," and connections are formed purely by chance. By studying the probability distributions of various centrality measures and the relations between them in this null model, we can identify which features of real networks are non-trivial and functionally significant.

Clustering Coefficient in Random Graphs. In an ER graph, the probability of an edge existing between any two nodes is uniformly equal to p, and every edge represents an independent event. Consequently, the probability that two neighbors of a given node i are connected is simply p.

Mathematically, the expected local clustering coefficient C_i can be derived directly from its definition:

$$C_i = \frac{k_i(k_i - 1) \cdot p}{k_i(k_i - 1)} = p$$

Since C_i is equal to p for every node, the global clustering coefficient C_G is also simply p. By expressing p in terms of the average network connectivity (mean degree) $\lambda = pN$, we obtain:

$$C_G = \frac{1}{N} \sum_i C_i = p = \frac{\lambda}{N}$$

This elegant result has profound topological implications in the thermodynamic limit. If we consider a family of ER graphs where the average degree λ is kept constant while the network size N increases to infinity, the clustering coefficient vanishes:

$$\lim_{\substack{N \to \infty \\ \lambda = \text{const}}} C_G = 0$$

This demonstrates that, in the thermodynamic limit, sparse random networks are **locally tree-like**. They contain virtually no triangles and no short cycles, unless the mean connectivity λ is forced to grow proportionally with the network size.

4.8.4 | Relationship Between Clustering and Degree

Because the local clustering coefficient C_i in an ER graph is identically p regardless of the node's degree k_i, there is absolutely no statistical correlation between C and K.

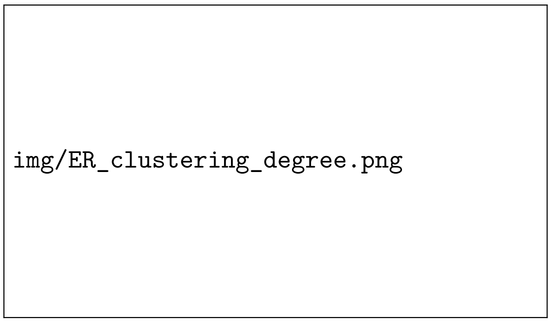

Figure 4.19: In an Erdős-Rényi random graph, the local clustering coefficient C_i is independent of the node degree k_i, resulting in a flat distribution when plotting C_i against k_i. This illustrates the lack of correlation between clustering and degree in random graphs.

However, in non-random, real-world networks, we observe a highly non-trivial relationship. For instance, in technological networks like the Internet, empirical data reveals a strong negative correlation between a node's degree and its clustering coefficient.

img/internet_clustering_degree.png

Figure 4.20: In the Internet, there is a strong negative correlation between node degree k and local clustering coefficient C_i. Highly connected hubs tend to have very low clustering coefficients, indicating that their neighbors are not interconnected. This reflects the role of hubs as global bridges connecting distinct communities.

In these complex systems, the most highly connected hubs tend to have very small clustering coefficients. This occurs because hubs act as global bridges connecting many distinct, otherwise disjoint sub-communities. Because the hub's neighbors belong to different functional or topological clusters, they are very unlikely to be connected to each other, thus suppressing the local transitivity C_i of the hub itself.

4.8.5 | Gaussianity of Centrality Measures in Random Graphs

Finally, we can analyze the probability density functions (PDF) of centrality measures in dense random graphs. Consider an ER graph with $N = 1000$ and $p = 0.1$, resulting in a high average degree of $\lambda = 100$.

While the exact degree distribution of an ER graph is binomial (which approaches a Poisson distribution for large N), for large values of λ, the Poisson distribution converges to a Gaussian (Normal) distribution by the Central Limit Theorem. We can visually verify this using a quantile-quantile (Q-Q) plot, where the empirical quantiles of the degree sequence are plotted against the theoretical quantiles of a Gaussian distribution.

img/ER_degree_qqplot.png

Figure 4.21: Q-Q plot for the degree distribution of an Erdős-Rényi graph with $N = 1000$ and $p = 0.1$. The points closely follow a straight diagonal line, confirming that the degree distribution is well-approximated by a Gaussian distribution in this regime.

Because nodes in an ER graph are structurally indistinguishable and the topology lacks complex correlations, local centrality measures tend to be tightly coupled to the node degree. As a result, measures like **Closeness Centrality** and **Eigenvector Centrality** also exhibit highly Gaussian distributions, forming near-perfect straight lines on their respective Q-Q plots.

img/ER_closeness_qqplot.png

Figure 4.22: Q-Q plot for Closeness Centrality, Eigenvector Centrality, and Betweenness Centrality illustrating their Gaussianity.

Anomalies in Betweenness: Interestingly, while Closeness and Eigenvector centralities are strongly Gaussian, **Betweenness Centrality** often shows noticeable deviations from the straight diagonal line, particularly at the tails of the distribution. This implies that even in a completely random topology, the non-local nature of shortest-path routing can generate extreme outlier values (bottlenecks) that do not strictly follow a normal distribution.

4.8.6 | Correlation Between Centrality Measures and Degree

In many standard network models, including random graphs, local centrality measures are not entirely independent of the node degree K. When evaluating these metrics across a network, we often observe strong correlations.

img/centrality_degree_correlation.png

Figure 4.23: Scatter plots showing the relationship between Degree (K) and various centrality measures (Betweenness Centrality, Closeness Centrality, Eigenvector Centrality) in a random graph. Closeness and Eigenvector Centrality exhibit a strong linear correlation with degree, while Betweenness Centrality shows a more complex, non-linear relationship.

As shown in the scatter plots, metrics such as **Closeness Centrality** and **Eigenvector Centrality** exhibit a strictly linear relationship with the degree K. In these topologies, knowing the degree of a node allows us to predict its Closeness or Eigenvector centrality almost perfectly. Consequently, these measures do not convey significantly "new" structural information about the nodes compared to the simple degree distribution.

However, **Betweenness Centrality (BC)** breaks this linear pattern. The relationship between BC and K is non-linear and much more sensitive to the macroscopic topology of the network.

The Upper Bound of Betweenness Centrality: $BC \propto K^2$

The non-linear relationship between Betweenness and Degree is mathematically bounded. Specifically, the Betweenness Centrality of a node grows at most quadratically with its degree:

$$BC \propto K^2$$

In general networks, this acts as a strict upper limit:

$$BC \leq \mathcal{O}(K^2)$$

To rigorously understand why this quadratic bound exists, we can consider the extreme topological case that maximizes Betweenness: a pure **tree structure** where the node in question acts as a central cutpoint.

`img/tree_cutpoint.png`

Figure 4.24: A tree structure where a central node v acts as a cutpoint connecting k distinct branches. This configuration maximizes the Betweenness Centrality of node v, as all shortest paths between nodes in different branches must pass through it.

Let v be a vertex with degree k. Assume that v acts as a perfect cutpoint connecting k distinct, otherwise disconnected sub-communities (branches). For simplicity, let's assume each of these k branches contains roughly N_0 nodes. Because the network is a tree, *any* shortest path traveling from a node in one branch to a node in another branch must strictly pass through v.

We can compute the exact Betweenness Centrality by combinatorial counting:

1. The number of possible pairs of branches is given by the binomial coefficient:
$$\binom{k}{2} = \frac{k(k-1)}{2}$$
2. For any given pair of branches, the number of distinct node pairs (one node in the first branch, one in the second) is:
$$N_0 \times N_0 = N_0^2$$

The total number of shortest paths passing through the cutpoint v is the product of these two quantities:

$$BC_v \approx N_0^2 \cdot \frac{k(k-1)}{2} \propto k^2$$

This derivation proves that $BC \propto K^2$ is exactly valid for a star-like tree configuration. In any real, non-tree network, the presence of cycles introduces alternative, redundant paths. These alternative routes allow some shortest paths to bypass the central node v, causing its actual Betweenness to drop below the theoretical maximum, which is why $BC \leq K^2$ serves as an upper bound.

4.8.7 | Hierarchy of Null Models

To conclude the analysis of network structures, it is important to note that the Erdős-Rényi (ER) graph is only the simplest baseline. Network science utilizes a hierarchy of null models to test hypotheses at different scales of complexity:

- **Global Null Models:** Models like the classical ER graph, where we hypothesize identical, homogeneous statistical properties for all nodes across the entire network.
- **Local Null Models:** Models where topological features change from node to node. The most famous example is the *Configuration Model*, which preserves the exact degree distribution (or degree sequence) of the real network but randomizes all connections. This allows us to test if a network's properties are simply a byproduct of its local degree sequence.
- **Meso-scale Null Models:** These intermediate models preserve structures at the community level. They assign connection probabilities based on the characteristics within network communities, or among nodes that share similar properties and structural roles (e.g., Stochastic Block Models).

5 | Diffusion Processes on Networks

5.1 | Fundamentals of Network Diffusion

In the study of dynamical processes on complex systems, network diffusion serves as a fundamental model for phenomena ranging from random walks and heat dissipation to the spreading of information or diseases. In this framework, the **nodes** of the network represent the possible states of the system (e.g., discrete positions in space, distinct chemical reaction compounds), while the **links** represent the allowed transitions or pathways between these states.

5.1.1 | Markov Chains and Transition Matrices

To mathematically describe the state of the system at any given time, we associate a value x_i to the i-th node, which represents the probability of the system being in the i-th state. Consequently, the global state of the network is described by a state vector $\vec{x}$, which must be a unit L^1 norm probability vector satisfying two conditions:

1. $x_i \geq 0$ for all i (non-negativity of probabilities).
2. $\sum_i x_i = 1$ (normalization).

The discrete-time evolution of this system is modeled as a **Markov Chain**. We define T_{ij} as the transition matrix element representing the probability of moving *from* node j *to* node i. The state vector updates at each discrete time step according to the linear relation:

$$x_i(t+1) = \sum_j T_{ij} x_j(t)$$

This equation states that the probability of finding the system at node i at time $t+1$ is exactly the sum of the probabilities entering node i from all possible nodes j at the previous time step t.

Because the system must go *somewhere* (including possibly staying at the same node), the probabilities of moving to any possible node i from a given state j must sum to unity:

$$\sum_i T_{ij} = 1$$

This property means the transition matrix T is *column-stochastic*. A direct consequence of this column normalization is the strict conservation of total probability over time:

$$\sum_i x_i(t+1) = \sum_i \sum_j T_{ij} x_j(t) = \sum_j \left(\sum_i T_{ij} \right) x_j(t) = \sum_j 1 \cdot x_j(t) = 1$$

Deriving the Transition Matrix from the Topology. In network science, the allowed transitions are dictated by the adjacency matrix A. To obtain the transition matrix T from A, we must rescale the columns of A by the total number of outgoing connections of each node. Let $K_j^{OUT} = \sum_i A_{ij}$ be the outgoing degree of node j. The elements of T are given by:

$$T_{ij} = \frac{A_{ij}}{K_j^{OUT}}$$

In matrix notation, we introduce a diagonal degree matrix D where the diagonal elements are $D_{ij} = \delta_{ij} K_i^{OUT}$. The transition matrix is then elegantly written as:

$$T = A \cdot D^{-1}$$

Important properties to note:

- Even if the underlying network is perfectly undirected (meaning the initial adjacency matrix A is symmetric), the resulting transition matrix T is generally **not symmetric** ($T_{ij} \neq T_{ji}$) unless the network is regular (i.e., all nodes share the exact same degree).
- Multiplying by D^{-1} from the *right* ($A \cdot D^{-1}$) normalizes the columns, which tracks forward probability flow. Conversely, if we were to multiply by D^{-1} from the *left* ($D^{-1} \cdot A$), we would normalize T by rows instead of columns. By multiplying a probability vector $\vec{x}(t)$ by this row-normalized matrix from the left, we can effectively **backpropagate diffusion in time**, a technique conceptually related to solving the backward Kolmogorov equation to identify the origin of a diffusion process.

5.1.2 | The Master Equation and the Diffusive Laplacian

While the Markov chain formalism describes discrete time steps, physical systems often evolve continuously. We can transition to a continuous-time formulation by defining a fundamental time step $\Delta t = 1$ and analyzing the rate of change of the probability vector:

$$\frac{\vec{x}(t+\Delta t) - \vec{x}(t)}{\Delta t} = (T - I) \cdot \vec{x}(t) = -(I - T) \cdot \vec{x}(t)$$

Taking the continuous limit yields the **Master Equation** of the diffusion process:

$$\dot{\vec{x}}(t) = -(I - T) \cdot \vec{x}(t) = -L_T \cdot \vec{x}(t)$$

Here, we have introduced $L_T = (I - T)$, which acts as the **"diffusion" Laplace operator** on the network.

We can express L_T explicitly in terms of the adjacency and degree matrices. By substituting $T = A \cdot D^{-1}$, we get:

$$L_T = I - A \cdot D^{-1} = (D - A) \cdot D^{-1}$$

It is crucial to distinguish this operator from the standard combinatorial Laplacian $L = (D - A)$. The diffusive Laplacian $L_T = L \cdot D^{-1}$ is the *random walk normalized Laplacian.* Because of the

D^{-1} term, it specifically governs conservative flow and probability diffusion on the graph geometry, ensuring that the total "mass" of the system remains invariant under the continuous time derivative ($\sum_i \dot{x}_i = 0$).

5.2 | Stationary States and Spectral Dynamics

By viewing network diffusion as a Markov chain, we can leverage the powerful tools of linear algebra and spectral graph theory to understand the long-term behavior of the system.

5.2.1 | The Eigenvalue Problem

The state of the system after N discrete time steps is obtained by applying the transition matrix T exactly N times to the initial state vector $\vec{x}(t_0)$. Mathematically, this corresponds to a matrix multiplication power:

$$\vec{x}(t_N) = T^N \vec{x}(t_0)$$

As $N \to \infty$, the system generally approaches a **stationary state** $\vec{x}_S$. A stationary state is defined as a state vector that remains unchanged by the evolution operator T:

$$\vec{x}_S = T\vec{x}_S$$

In the continuous-time framework (the master equation), this stationary state corresponds to a state of dynamic equilibrium where the time derivative vanishes ($\dot{\vec{x}} = 0$).

The search for the stationary state is naturally formulated as an **eigenvalue problem**:

$$T \cdot \vec{x} = \lambda \cdot \vec{x} \quad \text{with} \quad \lambda = 1$$

We are guaranteed that an eigenvalue of $\lambda = 1$ always exists for a column-stochastic transition matrix. This becomes evident when we look at the *left* eigenvectors. Let $\vec{1} = (1, 1, \ldots, 1)$ be the all-ones vector. Because the columns of T sum to 1 ($\sum_i T_{ij} = 1$), multiplying T from the left by $\vec{1}$ returns $\vec{1}$:

$$\vec{1}T = \vec{1}$$

Because T is generally asymmetric, its left and right eigenvectors are different. However, a fundamental property of matrices is that the left and right eigenvalue spectra are identical, ensuring that the right eigenvalue $\lambda = 1$ (and its corresponding stationary state eigenvector) also exists.

5.2.2 | Stationary States in Symmetric Networks

If the original adjacency matrix A of the network is symmetric ($A_{ij} = A_{ji}$), an exact analytical solution for the stationary state can be easily demonstrated. In such networks, the stationary state vector $\vec{x}_S$ has components that are strictly proportional to the node degree k:

$$\vec{x}_S \propto \vec{k}$$

Proof: We know that the node degree is $k_j = \sum_i A_{ij}$, and the transition matrix elements are defined as $T_{ij} = \frac{A_{ij}}{k_j}$. We want to test if the degree vector $\vec{k}$ satisfies the eigenvalue equation $T\vec{k} = \vec{k}$. Evaluating the i-th component of $T\vec{k}$ gives:

$$(T\vec{k})_i = \sum_j T_{ij} k_j = \sum_j \frac{A_{ij}}{k_j} k_j = \sum_j A_{ij}$$

Because the matrix A is symmetric, summing over the columns j of row i is identical to summing over the rows i of column j. Therefore, $\sum_j A_{ij} = \sum_j A_{ji} = k_i$. Thus, $T\vec{k} = \vec{k}$, proving that the degree vector is indeed the eigenvector corresponding to $\lambda = 1$.

5.2.3 | Transient Dynamics and Numerical Solutions

To understand how the system approaches this stationary state, we can decompose the initial state $\vec{x}(0)$ into a linear combination of the eigenvectors $\vec{x}^{(\lambda_i)}$ of the transition matrix:

$$\vec{x}(0) = \sum_i c_i \cdot \vec{x}^{(\lambda_i)}$$

Evolving this state forward in time by applying T^N yields:

$$\vec{x}(N\tau) = T^N \vec{x}(0) = \sum_i c_i \cdot \lambda_i^N \cdot \vec{x}^{(\lambda_i)}$$

We can factor out the dominant (maximum) eigenvalue λ_{MAX}^N:

$$\vec{x}(N\tau) = \lambda_{MAX}^N \sum_i c_i \cdot \left(\frac{\lambda_i}{\lambda_{MAX}} \right)^N \cdot \vec{x}^{(\lambda_i)}$$

Because λ_{MAX} is the largest eigenvalue, for all other eigenvectors $\lambda_i < \lambda_{MAX}$, the fraction $\frac{\lambda_i}{\lambda_{MAX}}$ is strictly less than 1. Consequently, as $N \to \infty$, the term $\left(\frac{\lambda_i}{\lambda_{MAX}}\right)^N \to 0$. These non-dominant eigenvectors represent **transient states** that decay geometrically in discrete time (or exponentially in the continuous limit). Eventually, only the component associated with λ_{MAX} survives, which is the stationary state.

Numerical Solutions (Power Iteration). For very large complex systems, solving the eigenvalue problem analytically becomes computationally unfeasible. Instead, numerical approximations are required. Based on the decay of transient states shown above, we can isolate the dominant eigenvector by repeatedly multiplying an arbitrary initial state vector $\vec{v}$ by the matrix A (or T).

$$\lim_{N\to\infty} A^N \cdot \frac{\vec{v}}{|\vec{v}|} = \vec{x}^{(\lambda_{MAX})}$$

Similarly, the dominant eigenvalue itself can be numerically extracted by taking the ratio of successive iterations:

$$\lim_{N\to\infty} \frac{A^{N+1} \cdot \vec{v}}{A^N \cdot \vec{v}} = \lambda_{MAX}$$

The Rayleigh Principle. Another profound way to frame the eigenvalue problem is through functional optimization using the **Rayleigh principle**. The dominant eigenvalue can be found by maximizing the Rayleigh functional:

$$\lambda_{MAX} = \max \left(\frac{\vec{x}^T \cdot A \cdot \vec{x}}{\vec{x}^T \cdot \vec{x}} \right)$$

The vector $\vec{x}$ that maximizes this quotient is precisely the dominant eigenvector $\vec{x}_{MAX}$. The remaining eigenvectors can be calculated analogously via constrained maximization, restricting the search space to the subspace orthogonal to the previously obtained eigenvectors ($\vec{x} \perp \vec{x}_\lambda$). Conversely, function minimization of the Rayleigh quotient will yield the smallest eigenvalue.

5.3 | The Perron-Frobenius Theorem

When dealing with the spectral dynamics of diffusion, two fundamental mathematical questions naturally arise:

1. Are we always guaranteed to find a maximal eigenvalue that is strictly unique (which would ensure a single, well-defined stationary state)?
2. What physical properties does the eigenvector associated with this maximal eigenvalue possess?

These questions are elegantly answered by the **Perron-Frobenius (P-F) Theorem**, a cornerstone of matrix algebra. The theorem is particularly powerful because it applies specifically to **non-negative matrices**—a class that perfectly encapsulates network graphs with positive link weights and transition probability matrices (Markov chains).

5.3.1 | Statement of the Theorem

The Perron-Frobenius theorem establishes the conditions under which a network will converge to a single, physically meaningful steady state.

Theorem Statement: Let A be a real, non-negative matrix that is both *irreducible* and *primitive*. Under these conditions:

1. A possesses a dominant, real eigenvalue λ_{MAX} that is strictly greater in absolute value than all other complex eigenvalues of the matrix: $\lambda_{MAX} > |\lambda|$.
2. The corresponding eigenvector $\vec{x}_{MAX}$ has strictly **real and positive values** ($x_i > 0\,\forall i$). This holds true even if the matrix A is completely asymmetric.
3. The eigenvalue λ_{MAX} has an algebraic multiplicity of exactly 1, meaning the maximal eigenvalue is unique.

In the context of network diffusion and Markov chains (where A becomes the transition matrix T), the dominant eigenvalue is $\lambda_{MAX} = 1$. The P-F theorem guarantees that the resulting stationary distribution vector $\vec{x}_S$ will not contain any negative or complex probabilities, nor will it assign a zero probability to any node, provided the topological conditions are met.

5.3.2 | Topological Conditions: Irreducibility and Primitivity

The theorem relies on two strict topological conditions: irreducibility and primitivity. Understanding when networks violate these conditions helps us understand pathological diffusion behaviors.

1. Irreducibility (Strong Connectivity)

A matrix is **irreducible** if and only if the underlying directed graph is *strongly connected*. This means that for every pair of nodes i and j, there exists at least one directed path from i to j, and vice versa.

If a network is *reducible*, it contains separated subsystems or "absorbing states." In linear algebra terms, a reducible matrix can be transformed via a basis change (using an invertible matrix B) into an upper block-triangular form:

$$\exists B \quad \text{such that} \quad B^{-1}AB = \begin{pmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{pmatrix}$$

Physically, this represents a network where probability mass or traffic can flow from the nodes in block A_{11} to the nodes in block A_{22} (via A_{12}), but can never return because the bottom-left block is strictly zero. Consequently, A_{22} acts as an absorbing basin or sink. In such cases, the network will completely drain into the absorbing states, and the corresponding stationary eigenvector will feature zero values for all nodes in the transient block A_{11}, violating the positivity condition of the P-F theorem.

2. Primitivity (Aperiodicity)

While irreducibility ensures there are no dead ends, **primitivity** ensures there are no infinite oscillatory loops. A primitive matrix is one that is non-negative, irreducible, and *aperiodic*.

A matrix fails to be primitive (it is *imprimitive*) if it represents a perfectly multipartite graph—groups of nodes that only connect to other distinct groups in a strict cycle. Imprimitivity implies that the dominant eigenvalue is not strictly unique in magnitude; there are multiple eigenvalues on the spectral circle ($\#(\lambda_{MAX}) > 1$ in terms of absolute value).

A classic example of an imprimitive matrix takes a cyclic block form:

$$A_{cyclic} = \begin{pmatrix} 0 & A_1 & 0 \\ 0 & 0 & A_2 \\ A_3 & 0 & 0 \end{pmatrix}$$

In a diffusion process on this topology, the system never settles into a stationary distribution. Instead, the probability mass shifts entirely from block 1, to block 2, to block 3, and back to block 1, infinitely. These "cyclic" solutions mean the state values "switch" forever between components, violating the convergence property. Additionally, an imprimitive matrix does not satisfy $A^K = A$ for any macroscopic power due to its periodicity.

Conclusion for Transition Matrices. For a Markov chain transition matrix to behave nicely, we must ensure it is both irreducible (no absorbing sinks or isolated subsystems) and primitive (no cyclic communicating blocks). When these two conditions are satisfied, the Perron-Frobenius theorem ensures the existence of a **unique stationary distribution** strictly greater than zero across the entire network.

5.4 | Eigenvector Centrality Variations: PageRank

A highly impactful variation of eigenvalue centrality is the **PageRank** algorithm, famously developed by Brin and Page in 1998 to rank website relevance for the Google search engine. In essence, the PageRank score of a website is mathematically defined as the stationary state of a diffusion process on the World Wide Web, which is modeled as a massive directed network of hyperlinks.

5.4.1 | The Random Surfer and the Damping Factor

In a directed network, relying purely on standard eigenvector centrality or simple diffusion is problematic. The hyperlink network of the web is not guaranteed to be strongly connected; it contains many isolated components, "sink" nodes with no outgoing links, and cyclic traps. To solve this, PageRank introduces a "smart" additional term.

The conceptual model is that of a "random surfer." The surfer navigates the web by randomly clicking on available outgoing links. The PageRank $PR(A)$ of a node (page) A depends on the PageRank of the k nodes pointing to it, scaled inversely by the number of outgoing links $C(k)$ from each of those k nodes. In other words:

- The higher the PageRank of the webpages pointing at you, the better.
- However, if a highly-ranked webpage points to many different pages, its conveyed importance is linearly diluted among all its targets.

To prevent the surfer from getting stuck in absorbing states or disconnected components, we introduce a **damping factor**, d (typically set to 0.85). With probability $1-d$, the surfer abandons the current link structure and "teleports" to a completely random page anywhere on the web.

The formula for the PageRank score of node A is written as:

$$PR(A) = (1-d) + d\left(\sum_k \frac{PR(k)}{C(k)}\right)$$

In vector and matrix notation, letting $\vec{x}$ be the vector of PageRank scores (where $\sum_i x_i = 1$) and T be the column-normalized transition matrix derived from the adjacency matrix, we can express this as:

$$\vec{x} = d \cdot T\vec{x} + (1-d)\vec{1}$$

Ensuring Perron-Frobenius Conditions. Because we must ensure the network is strongly connected to find a valid stationary state, the algorithm employs a clever mathematical trick. We define a completely connected matrix C, where every single element is 1. The "teleportation" term can be represented using this matrix. Since $\vec{x}$ is a probability vector ($\sum_i x_i = 1$), multiplying it by $\frac{C}{N}$ returns the all-ones vector scaled by $\frac{1}{N}$:

$$\frac{C}{N} \cdot \vec{x} = \vec{1} \cdot \frac{1}{N}$$

(Note: The slides use $(1-d)\vec{1}$ loosely to represent the uniform redistribution of probability, which mathematically acts as $(1-d)\frac{C}{N}\vec{x}$ in the closed linear system).

By combining the topological transitions and the random jumps, the system evolves according to a new, effective transition matrix:

$$\vec{x} = \left(d \cdot T + (1-d)\frac{C}{N} \right) \vec{x}$$

The addition of the uniform connectivity matrix C guarantees that every node is at least weakly connected to every other node. Therefore, the new effective transition matrix is strictly positive, satisfying both the **irreducibility** and **primitivity** conditions. By the Perron-Frobenius theorem, this guarantees a unique stationary state solution where all PageRank scores are strictly positive, corresponding to the maximum eigenvalue.

5.4.2 | Analytical and Numerical Formulation

To find the stationary state $\vec{x}$, we can attempt an analytical solution. Starting from $\vec{x} = d \cdot T\vec{x} + (1-d)\vec{1}$, we can rearrange the terms:

$$(I - d \cdot T)\vec{x} = (1-d)\vec{1}$$

By inverting the matrix $(I - d \cdot T)$, we obtain the exact analytical solution:

$$\vec{x} = (1-d)(I - d \cdot T)^{-1}\vec{1}$$

The positive values of the resulting eigenvector $\vec{x}$ (guaranteed by the P-F theorem) are the definitive webpage ranking scores.

The Need for Numerical Iteration. While mathematically elegant, calculating the exact analytical solution via matrix inversion is computationally unfeasible in practice. Matrix inversion has a complexity of $\mathcal{O}(N^3)$, and the network of the World Wide Web consists of billions of websites.

The standard solution is to **numerically iterate** the diffusion process. We define the iterative step using the effective transition matrix $P = d \cdot T + (1-d)\frac{C}{N}$:

$$\vec{x}(t+1) = P \cdot \vec{x}(t)$$

Starting from an initial random guess $\vec{x}(0)$, we apply P repeatedly. After a sufficient number of steps $N \gg 1$, the transient states decay, and the vector gets closer to the convergence of the stationary state $\vec{x}_S$:

$$\vec{x}_S = \lim_{N \to \infty} P^N \vec{x}(0)$$

The speed at which this numerical approach converges to the stationary state depends heavily on the spectral gap—specifically, the relationship between the dominant eigenvalue (which is 1) and the second largest eigenvalue of the matrix P.

5.5 | Random Walk Betweenness

Standard Betweenness Centrality (BC) is built on a very strong and somewhat idealized assumption: that information, traffic, or physical flow in a network always travels exclusively along the absolute shortest paths. While mathematically elegant, this assumption is often violated in real-world scenarios. In systems like vehicular traffic, the internet, or social gossip, agents may lack perfect global knowledge of the network or intentionally choose slightly longer, alternative routes to avoid congestion.

To capture a more realistic model of flow, we introduce **Random Walk Betweenness (RWB)**. This centrality measure evaluates the importance of a node based on the expected number of times it is traversed by a random walk. Because a random walker explores the network probabilistically, RWB considers *all* possible paths, not just the shortest ones.

5.5.1 | Pathological Cases of Standard Betweenness

The limitations of shortest-path BC become starkly apparent in certain "pathological" topological configurations, where near-optimal alternative roads are completely ignored.

img/pathological_BC.png

Figure 5.1: Pathological cases in Betweenness Centrality: (a) The Triangle Bridge and (b) Parallel Bridges.

Consider the two configurations in the referenced figure:

- **Configuration (a) - The Triangle Bridge:** Two dense communities (Group 1 and Group 2) are connected by three nodes, A, B, and C, arranged in a triangle. Nodes A and B are directly connected to the groups and to each other. Because the direct path $A \to B$ is length 1, the path $A \to C \to B$ (length 2) is strictly sub-optimal. Consequently, nodes A and B will accumulate massive standard BC scores, while node C will have a BC of exactly zero. Yet, structurally, C provides a highly relevant alternative bridge.

- **Configuration (b) - Parallel Bridges:** In cases of multiple parallel links between communities, BC can heavily skew towards one specific path depending on minute weight differences, entirely discounting redundant, slightly longer paths.

Random Walk Betweenness resolves this. By comparing a node's standard BC with its Random Walk BC (often using a scatterplot of the two measures or analyzing their numerical difference), one can easily identify these hidden "alternative route" nodes that standard algorithms miss.

5.5.2 | Markov Chains and Walk Probabilities

To formalize Random Walk Betweenness, we model the diffusion process as a Markov chain. Let us define a random walk transition matrix M, where M_{ij} represents the probability of moving *from* node j *to* node i in a single step. If the network is unweighted, this is simply the inverse of the outgoing degree:

$$M_{ij} = p(i|j) = \frac{A_{ij}}{k_j}$$

By definition, the sum over all possible destinations i from a given origin j is normalized: $\sum_i M_{ij} = 1$.

If $M_{ij} = 0$, a direct 1-step transition from j to i is impossible. However, the walker could reach i in 2 steps by passing through an intermediate node k (i.e., $j \to k \to i$). The probability of this specific 2-step path is $M_{ik} \cdot M_{kj}$. To find the total probability of transitioning from j to i in exactly 2 steps across *all* possible intermediate nodes, we sum these products, which is simply the matrix multiplication M^2:

$$[M^2]_{ij} = \sum_k M_{ik} \cdot M_{kj}$$

By induction, the probability of going from j to i in exactly N steps is proportional to the ij-th element of the matrix M^N:

$$P(j \to i \text{ in } N \text{ steps}) \propto [M^N]_{ij}$$

Intrinsic Path Weighting. This formulation inherently solves the problem of weighting paths by their length. Because the transition probabilities are fractional ($M_{ij} < 1$ for nodes with degree > 1), the elements of M^N become progressively smaller as N increases. Therefore, while RWB counts all possible walks, very long, meandering walks contribute exponentially less to the centrality score than short, direct paths. The walks are automatically weighted by their length.

Infinite Random Walks and Convergence. To compute the total expected number of passages through a target node j when starting a random walk from a source node s, we must sum the probabilities over all possible time steps r from 0 to infinity:

$$\sum_{r=0}^{\infty} P(j|t=r) = \sum_{r=0}^{\infty} [M^r]_{js}$$

This is a geometric series of matrices. Because the magnitude of the eigenvalues of the strictly transient parts of the random walk matrix M is strictly less than 1 ($|\lambda| < 1$), this matrix geometric series converges:

$$\sum_{r=0}^{\infty} M^r = (I - M)^{-1}$$

Thus, the total expected passages through j from s is given by the js-th entry of the matrix $(I - M)^{-1}$.

Note for future derivation: If we want to consider the probability of the walker making one further movement exiting node j, we can multiply this result by $\frac{1}{k_j}$. This mathematical trick will become highly relevant when linking this concept to the graph Laplacian.

5.5.3 | Absorbing States and the Laplacian Pseudoinverse

To calculate the expected number of times a random walker passes through a specific node, we evaluate the infinite sum of the transition matrix powers, $\sum_{r=0}^{\infty} M^r = (I - M)^{-1}$. This infinite geometric series is deeply connected to the network's Laplacian matrix.

By substituting the unweighted transition matrix definition $M = AD^{-1}$ (where A is the adjacency matrix and D is the diagonal degree matrix), we can manipulate the inverse expression:

$$(I - M)^{-1} = (I - AD^{-1})^{-1} = \left((D - A)D^{-1}\right)^{-1} = D(D - A)^{-1} = DL^{-1}$$

Here, $L = D - A$ represents the standard combinatorial graph Laplacian. If we consider an additional "exit" step from the target node j, which occurs with probability $1/k_j$, we effectively multiply the right side by D^{-1}. This elegantly reduces the equation exactly to the inverse Laplacian L^{-1}. Thus, the total number of passages through a node during an infinite random walk is fundamentally governed by the inverse of the Laplacian matrix.

The Singularity Problem and the Absorbing State Trick. A critical mathematical issue arises here: the Laplacian matrix L always has at least one eigenvalue equal to zero ($\lambda_1 = 0$, corresponding to the all-ones eigenvector $\vec{1}$), meaning its determinant is zero ($|L| = 0$) and the matrix is strictly non-invertible.

To bypass this singularity, we can use the **absorbing state trick**. We define our destination node t as an "absorbing state", meaning that once the random walker reaches t, it can never leave. Mathematically, we force all transition probabilities out of t to zero:

$$M_{it} = 0 \quad \forall i$$

In matrix terms, this corresponds to deleting the t-th row and t-th column from the transition matrix (or the Laplacian), resulting in a "reduced" sub-matrix M_t (or L_t). Because the zero-eigenvalue is removed along with the grounded node, this reduced matrix *is* invertible. We can calculate the random walk passages through any intermediate node using this inverted reduced matrix, and simply insert zeros back for the t-th row and column afterward. By summing these values over all possible source-target pairs (s, t), we obtain the Random Walk Betweenness for edges and nodes.

The Moore-Penrose Pseudoinverse. Alternatively, instead of grounding a specific node, we can compute the **pseudoinverse** (often denoted as L' or L^+) using spectral decomposition. Given the eigenvalues λ_i and the corresponding orthonormal eigenvectors $\vec{u}_i$ of the symmetric Laplacian L, the matrix can be projected as:

$$L = \sum_{i=1}^{N} \lambda_i \cdot \vec{u}_i \cdot \vec{u}_i^T$$

Because the singularity is caused solely by the first eigenvalue $\lambda_1 = 0$, the pseudoinverse is constructed by simply skipping the first term and inverting the non-zero eigenvalues:

$$L' = \sum_{i=2}^{N} \frac{1}{\lambda_i} \cdot \vec{u}_i \cdot \vec{u}_i^T$$

This pseudoinverse L' allows us to handle random walk calculations symmetrically without artificially removing nodes.

5.5.4 | Hitting Time and Commute Time Distance

The pseudoinverse provides a powerful framework for defining topological distances based on random walks, moving beyond simple shortest-path geodesics.

We define the **Hitting Time** H_{ij} as the expected (average) number of random jumps required for a walker to travel from node i to node j. Note that in general networks, hitting time is asymmetric ($H_{ij} \neq H_{ji}$) because it might be "downhill" (easier) to reach a hub from a leaf than vice versa.

To create a proper, symmetric metric, we define the **Commute Time Distance (CTD)** C_{ij}, which is the expected number of jumps to walk from i to j *and back* to i:

$$C_{ij} = H_{ij} + H_{ji}$$

Using the Laplacian pseudoinverse L', the Commute Time Distance can be calculated analytically. Let $\vec{e}_i = [0, \dots, 1, \dots, 0]^T$ be the standard basis vector for node i. The CTD is proportional to the quadratic form of the pseudoinverse:

$$C_{ij} = \left(\sum_k k_k \right) \cdot (\vec{e}_i - \vec{e}_j)^T L' (\vec{e}_i - \vec{e}_j)$$

where $\sum_k k_k$ is the total volume of the network degrees.

CTD as a Euclidean Embedding. A profound consequence of this mathematical structure is that the Commute Time Distance acts as a pure squared Euclidean distance in a transformed space. By decomposing the pseudoinverse, we can map every node i to a coordinate vector $\vec{\Psi}_i$ in an $(N-1)$-dimensional Euclidean space:

$$\vec{\Psi}_i = \sqrt{\sum_k k_k} \cdot (\Lambda')^{-1/2} U^T \vec{e}_i$$

where Λ' is the diagonal matrix of inverted non-zero eigenvalues and U contains the eigenvectors.

In this new embedding space, the Commute Time Distance between any two nodes is exactly their squared Euclidean distance:

$$C_{ij} = ||\vec{\Psi}_i - \vec{\Psi}_j||^2$$

Nodes that are deeply connected by many redundant short paths (e.g., within the same dense community) will be clustered very close together in this $\vec{\Psi}$ space, whereas nodes separated by bottlenecks will be pushed far apart.

Physical Analogy. The mathematical framework of Random Walk Betweenness is profoundly connected to the laws of electromagnetism. In fact, calculating the expected number of random walk passages through a node is mathematically equivalent to calculating the current flowing through a component in an electrical circuit, where edges act as resistors. The full derivation of this equivalence, known as **Current Flow Centrality**, is provided in Appendix A.

6 | The Laplacian Operator and Heat Kernel Formalism

7 | Scale-Free Networks and Generative Models

7.1 | Introduction to Scale-Free Networks

7.1.1 | Heavy-Tailed Degree Distributions

7.1.2 | Empirical Examples

7.2 | The Barabási-Albert (BA) Model

7.2.1 | Core Mechanisms: Growth and Preferential Attachment

7.2.2 | Continuum Theory and Rate Equation

7.2.3 | Topological Properties and Limitations of the BA Model

7.3 | Extensions of the Barabási-Albert Model

7.3.1 | Non-linear Preferential Attachment

7.3.2 | Node Fitness and Competitive Growth

7.4 | The Duplication-Divergence Model

7.4.1 | Biological Motivation: Gene Duplication

7.4.2 | Mechanisms of Duplication and Divergence

7.4.3 | Mathematical Formulation and Tunable Exponents

7.5 | Implicit Preferential Attachment and Analogies

7.5.1 | The Chinese Restaurant Process (CRP)

7.5.2 | Emergence of Implicit Preferential Attachment

Appendices

A | Electrical Circuit Analogy and Current Flow Centrality

The Diffusive Laplacian and Random Walk Betweenness can be elegantly interpreted using the physical framework of electrical circuits. In this macroscopic analogy, the network topology forms the wiring of a circuit, where edges act as conductive wires and nodes act as junction points.

A.1 | Networks as Electrical Circuits

Let us consider an electrical circuit mapped onto a network with N nodes and L links. Each node i is assigned an electrical potential V_i. By the laws of physics, absolute potential is arbitrary; only potential *differences* (ΔV) drive currents. Mathematically, this means that the vector of potentials $\vec{V}$ is known only up to an additive constant c, which corresponds to the fact that the constant vector $\vec{x} = c(1, 1, \ldots, 1)$ lies in the null space of the system.

The relation between the node potentials and the currents flowing through the links is governed by the **Incidence Matrix** I. The incidence matrix is an $L \times N$ matrix that acts as a discrete derivative operator (analogous to the gradient $\vec{\nabla}$) on the graph. For each directed edge k going from node i to node j, the matrix calculates the finite difference between neighboring nodes. The orientation of the derivative (+ and - signs) corresponds to an arbitrary initial choice for the verse of the currents.

By Ohm's law, if we assume unit conductance for all edges initially, the current i_k passing through edge k is simply the potential difference between its endpoints:

$$i_k = \sum_j I_{kj} V_j \implies \vec{i} = I \cdot \vec{V}$$

In a more realistic weighted network, links can have different conductances (the inverse of resistance, $g = 1/R$). We define an $L \times L$ diagonal matrix G, where the diagonal elements $G_{kk} = g_k$ provide the specific weight (conductance) for each link. If we also consider external currents $\vec{i}_{EXT}$ that are externally injected into specific nodes, the generalized equation for the edge currents becomes:

$$\vec{i} = \vec{i}_{EXT} - G \cdot I \cdot \vec{V}$$

A.2 | Kirchhoff's Laws and the Graph Laplacian

While the incidence matrix I maps node potentials to edge currents, its transpose I^T (dimension $N \times L$) performs the inverse topological operation: it maps edge currents back to the nodes.

Specifically, the i-th row of I^T sums all the currents passing through the i-th node.

According to **Kirchhoff's Current Law** (which dictates the conservation of total charge), the net current accumulating at any node must be zero (unless it is explicitly an external source or sink):

$$I^T \cdot \vec{i} = 0$$

We can now substitute the topological Ohm's law expression for $\vec{i}$ into Kirchhoff's law to link the external currents directly to the node potentials:

$$I^T \cdot (\vec{i}_{EXT} - G \cdot I \cdot \vec{V}) = 0$$

$$I^T \cdot \vec{i}_{EXT} = (I^T \cdot G \cdot I) \cdot \vec{V}$$

The central operator $(I^T \cdot G \cdot I)$ is mathematically identical to the **Weighted Graph Laplacian**, L_W. Therefore, the global equation governing the steady state of the entire circuit simplifies to:

$$I^T \cdot \vec{i}_{EXT} = L_W \cdot \vec{V}$$

Solving for Potentials (Grounding). To find the explicit potentials $\vec{V}$, we need to invert the Laplacian matrix. However, as noted earlier, the pure Laplacian has a zero eigenvalue associated with the constant vector $\vec{1}$, meaning its determinant is zero ($|L_W| = 0$) and it cannot be natively inverted.

In electrical engineering, this singularity is resolved physically by **grounding** a node (e.g., setting the potential of a specific node t to $V_t = 0$). Mathematically, this corresponds to removing the t-th row and column from the matrix L_W. The resulting reduced matrix, L'_W, is strictly positive definite and invertible. We can then perfectly solve for the relative potentials of the remaining nodes:

$$\vec{V}_{reduced} = (L'_W)^{-1} \cdot (I^T \cdot \vec{i}_{EXT})_{reduced}$$

This procedure of grounding a node to solve the electrical potential field is mathematically isomorphic to the "absorbing state trick" used to invert the transition matrix when calculating the Random Walk Betweenness for a single node.

A.3 | Node-Level Formulation and the Adjacency Matrix

While the incidence matrix provides a macroscopic topological view, we can formulate Kirchhoff's laws at the local node level using the network's **Adjacency Matrix** (A).

Let us consider a specific problem: injecting a unit of current into a source node s ($s = in$) and extracting it from a sink node t ($t = out$).

img/bridge_circuit.png

Figure 1: An example of a network represented as an electrical circuit, with a source node s and a sink node t.

Assuming unit conductances for the links (where resistance $R_{ij} = 1$, making $A_{ij} = 1/R_{ij} = G_{ij}$), the current flowing between any two connected nodes i and j is simply their potential difference:

$$i_{ij} = \frac{\Delta V_{ij}}{R_{ij}} = A_{ij}(V_i - V_j)$$

Note: In a weighted network, the edge weights naturally represent different conductance values (or equivalently, different inverse resistances).

By Kirchhoff's law of charge conservation, the sum of all currents entering and leaving any intermediate node must be exactly zero. The only exceptions are the source s (where 1 unit enters) and the sink t (where 1 unit leaves). We can write this elegantly using the Kronecker delta:

$$\sum_j i_{ij} = 0 \implies \sum_j A_{ij}(V_i - V_j) = \delta_{is} - \delta_{it}$$

A.4 | Deriving the Laplacian and Solving for Potentials

We can mathematically expand the sum of currents to reveal the graph Laplacian:

$$i_i = \sum_j A_{ij}(V_i - V_j) = \sum_j A_{ij}V_i - \sum_j A_{ij}V_j$$

Because $\sum_j A_{ij} = k_i$ (the node degree), we can factor out V_j using the Kronecker delta:

$$i_i = \sum_j (k_i \cdot \delta_{ij} - A_{ij}) \cdot V_j$$

This term in parentheses is the exact definition of the **Combinatorial Laplacian** $L = D - A$. In vector notation, this confirms our previous derivation: $L\vec{V} = \vec{s}$, where $\vec{s}$ is the external current vector $[0, \dots, 1, \dots, -1, \dots, 0]^T$.

As established, the matrix L cannot be inverted because $|L| = 0$ (its null space contains the all-ones vector $\vec{x} = (1, 1, \dots, 1)$, physically meaning that potentials are only known up to an arbitrary additive constant).

To solve the system, we apply the boundary conditions by **"grounding"** the sink node t, setting $V_t = 0$. We remove the t-th row and column from L to obtain the reduced, invertible matrix L_t. The potentials of the remaining nodes are then strictly determined:

$$\vec{V}^{(st)} = L_t^{-1} \cdot \vec{s}_t$$

A.5 | Exact Equivalence: Current Flow Centrality

Once the potentials $V^{(st)}$ are known, we can calculate the absolute current passing through any node i. The absolute current through i is defined as half the sum of the absolute currents on all its incident links (to avoid double counting incoming and outgoing flows):

$$i_i^{(st)} = \frac{1}{2} \sum_j A_{ij} |V_i^{(st)} - V_j^{(st)}|$$

The **Current Flow Centrality (CFC)** of node i is then defined as the total current passing through it, summed over all possible source-sink pairs (s, t) in the network:

$$CFC_i = \sum_{s \neq t} i_i^{(st)}$$

Connecting Random Walks and Circuits. The profound beauty of this network measure lies in its translation back to Random Walk probabilities. The calculated electrical potentials $V^{(st)}$ can be explicitly rewritten in terms of the network's Hitting Times (T), which are derived from the pseudoinverse of the Diffusive Laplacian.

By substituting the potential differences with hitting times, the current through node i exactly matches the random walk passages formulation:

$$i_i^{(st)} = \frac{1}{2} \sum_j A_{ij} |T_{is} - T_{it} - T_{js} + T_{jt}| \quad \text{for } i \neq s, t$$

Starting from two entirely distinct physical processes—an absorbing Markov chain and the continuous flow of current in a circuit—we arrive at the exact same topological measure of centrality. In fact, physical current flow can be fundamentally understood as a "charged particle" executing a random walk through the network, where the voltage differences generated by the source and ground provide a directional drift.